

**A
MAJOR PROJECT REPORT
ON
“Recipe Finder Website”**

**Submitted in the partial fulfillment of the requirement
for the award of the degree of**

**BACHLOR OF TECHNOLOGY
OF
COMPUTER SCIENCE AND ENGINEERING**



Supervision of :

Dr. Ajay Kumar
Assts.Professor
Dept. of CSE

Submitted by:

Ombir
220011015046
Dept. of CSE

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
INDIRA GANDHI UNIVERSITY MEERPUR, REWARI
Session (2022-2026)**

I
INDIRA GANDHI UNIVERSITY MEERPUR REWARI
DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
CERTIFICATE OF COMPLETION

This is to certify that the major project entitled “RECIPE FINDER WEBSITE” carried out by Ombir, student of Bachelor of Technology in Computer Science and Engineering 8th Semester, of Indira Gandhi University, Meerpur, Rewari is a satisfactory account of the bonafide work under my supervision, is recommended towards the end of his 8th semester of B.Tech (CSE).

Project Incharge:

Dr. Ajay Kumar

(Assistant professor)

II

DECLARATION BY THE CANDIDATE

I, Ombir, hereby declare that the major project work entitled “RECIPE FINDER WEBSITE” is an authenticated work carried out by me at Indira Gandhi University for the partial fulfillment of the award of the degree of B.Tech, Indira Gandhi University, Meerpur, Rewari and this work has not been submitted for similar purpose anywhere else.

Date:

Place: IGU, Meerpur

Ombir

220011015046

B.Tech 8th SEM

III

ACKNOWLEDGEMENT

It is a genuine pleasure to express my profound gratitude and deep regards to my guide “Dr. Ajay Kumar”, Assistant Professor in CSE Department, for his exemplary guidance, monitoring and constant encouragement throughout this project. His insights and vast experience in the field of web technologies and software engineering have been invaluable in helping me achieve the goals of this project.

I would like to express my special thanks to ‘Indira Gandhi University’ who gave me the golden opportunity to do this wonderful project on the topic “RECIPE FINDER WEBSITE”, which helped me in doing a lot of research and I came to know about so many new things related to web development, APIs, and user experience design.

I am also grateful to all the faculty members of the Department of Computer Science & Engineering for their support and guidance. I also express my sincere thanks to my family and friends for their unwavering support and encouragement throughout the course of this project.

Ombir
220011015046
B.Tech 8th SEM

IV

List of Figures

Figure 1.1: Recipe Finder Website Overview	2
Figure 1.2: Recipe Finder Schematic Diagram.....	6
Figure 2.1: Spoonacular API Integration.....	10
Figure 2.3: Architecture of Recipe Finder.....	15
Figure 4.2: Website Process Flow Diagram	28
Figure 4.2: Data Flow Diagram-2.....	29
Figure 5.1: Life of a Search Query	35
Figure 5.2: Frontend Architecture	38
Figure 5.3: Backend Memory Management.....	41
Figure 6: Designing the Recipe Finder.....	48
Figure 6.1: Level-0 of DFD.....	50
Figure 6.1: Level-1 of DFD.....	51
Figure 6.1.1: Dataflow Diagram.....	53
Figure 6.2.1: Use Case Diagram.....	54
Figure 6.2.2: Class Diagram	55
Figure 6.2.3: Sequence Diagram	56
Figure 6.2.4: Activity Diagram	58
Figure 8.2.1: Unit Testing	67
Figure 8.3.1: Black Box Testing.....	70
Figure 8.3.2: White Box Testing.....	71

ABSTRACT

In today's fast-paced digital world, people increasingly rely on technology to assist with everyday activities, including meal planning and cooking. This project presents the design and development of a Recipe Finder Website, a full-stack web application built using HTML, CSS, JavaScript (Frontend) and Node.js with Express.js (Backend), integrated with the Spoonacular API to deliver intelligent, real-time recipe discovery and detailed nutritional information to users.

The Recipe Finder Website is designed to allow users to search for recipes based on ingredients they have at hand, dietary restrictions, cuisine types, or specific dish names. It leverages the power of the Spoonacular API to fetch comprehensive recipe data, including ingredients, cooking steps, preparation time, calorie counts, and nutritional breakdowns. The system is accessible through a user-friendly, responsive interface and is capable of handling multiple simultaneous search queries with minimal latency.

Key features of the system include: ingredient-based search, filter options by diet (vegan, vegetarian, gluten-free, etc.), cuisine type filtering, recipe detail view with step-by-step instructions, nutritional information display, and a favorites/bookmarking system for registered users. The application also incorporates a user authentication module, allowing personalized recipe recommendations and history tracking.

This project demonstrates how modern web technologies and open food APIs can be effectively utilized to improve user experience in recipe discovery, reduce meal planning effort, and promote healthy eating habits. Through this implementation, we aim to showcase the potential of web-based applications in transforming everyday cooking and dietary decision-making experiences across various demographics.

Table of Contents

Sr. No.	Chapter Title	Page No.
1	Certificate of Completion	i
2	Declaration by the Candidate	ii
3	Acknowledgement	iii
4	List of Figures	iv
5	Abstract	V
6	Introduction	1
7	Technological Environment (System Analysis & Architecture)	7-20
8	Feasibility Report	21
9	Software Requirements Specification (SRS)	23
10	Introduction to Spoonacular API & Frontend Framework	34
11	System Design	45
12	Entity Relationship (ER Diagram)	48
13	Data Flow Diagrams (DFDs)	50-51
14	UML Diagrams (Class, Sequence, Activity)	54-58
15	Output Screens	59-64
16	System Testing (Unit Testing, Integration Testing)	65-73
17	Conclusion and Future Scope	74-75
18	Coding (Implementation)	76-84
19	Abbreviations	85-87

6

INTRODUCTION

INTRODUCTION TO PROJECT

- A Recipe Finder Website is a web-based application that helps users discover, search, and explore recipes from around the world.
- It is a comprehensive platform that connects food enthusiasts with an extensive database of recipes, providing step-by-step cooking instructions, nutritional information, and ingredient lists.
- The application uses modern web technologies and integrates with third-party food APIs to deliver real-time recipe data to users.
- The term Recipe Finder encompasses search-by-ingredient functionality, cuisine-based filtering, dietary restriction filters (vegan, vegetarian, gluten-free), and personalized meal planning suggestions.
- The website can interact with users through different channels such as web browsers on desktops, tablets, and mobile devices.
- Users spend significant time searching for recipes online, making the internet the most popular platform for recipe discovery and meal planning. Therefore, a well-designed Recipe Finder Website can provide tremendous value to daily users.

The development of a Recipe Finder Website project requires a thorough understanding of the needs and preferences of the target users – home cooks, professional chefs, dietary-conscious individuals, and food enthusiasts. Developers must analyze user search patterns and design intuitive interfaces that accurately respond to their culinary queries. Additionally, the system must integrate with reliable food databases and should be able to learn from user feedback to improve recommendations over time.

A Recipe Finder Website project involves developing a software application that can communicate with users through a web interface, providing them with relevant recipe information and cooking guidance. The purpose of such a project is to simulate a personalized digital cookbook experience and provide users with relevant recipes based on their preferences, available ingredients, and dietary needs. Recipe Finder projects can be used in various domains, including home cooking, professional kitchens, nutrition management, and food education.

Fig 1.1: Recipe Finder Website Benefits

[Figure 1.1: Infographic showing Recipe Finder benefits – 40% users prefer online recipes, 5x faster meal planning, instant ingredient-based search]



40% users prefer online recipes

A large portion of users prefer finding recipes online for convenience and variety.



5x faster meal planning

Plan meals up to 5 times faster with easy access to recipes, filters, and recommendations.



Instant ingredient-based search

Find recipes instantly using the ingredients you already have at home.



Encourages home cooking

Inspires users to cook healthy and delicious meals at home.



Supports nutrition education

Promotes awareness of healthy eating and balanced nutrition through recipes.

Existing System

Several existing systems for recipe finding have already been developed and are in use by various organizations and individuals. These existing systems provide a solid foundation for building recipe platforms, and can help speed up development by providing pre-built modules and APIs. However, it is important to note that each system has its own set of strengths and weaknesses, and no single system is ideal for every situation. Thus, it is important to carefully consider the requirements of the project and choose the system that best meets the needs.

Here are some examples of existing recipe platforms:

- Allrecipes.com – One of the world's largest recipe platforms with community-submitted recipes, user reviews, and personalized recommendations based on browsing history.
- Yummly – A smart recipe recommendation platform that provides personalized recipe suggestions based on taste preferences, dietary restrictions, and cooking history.
- Food.com – A community-driven recipe website featuring millions of recipes submitted by home cooks and professional chefs with rating and review systems.
- Epicurious – A premium recipe platform by Conde Nast that offers curated recipes from top culinary magazines like Bon Appétit and Gourmet.
- Spoonacular API – A powerful food and recipe API used by developers to integrate recipe search, nutrition information, and meal planning features into their applications.

Proposed System

The proposed system for the Recipe Finder Website project is a modular web architecture that can be adapted to different user needs (Fig 1.2: Recipe Finder Schematic Diagram). The system consists of four main components:

- Search Engine Module: The search module is responsible for processing user input and querying the Spoonacular API. It breaks down user queries into parameters such as ingredients, cuisine, diet type, and cooking time to retrieve the most relevant recipes.

Recipe Management Module (Module-1): This module handles recipe data retrieval, parsing, and display. It fetches recipe details including ingredients, cooking steps, preparation time, serving size, and nutritional information from the API and presents them in a structured, user-friendly format.

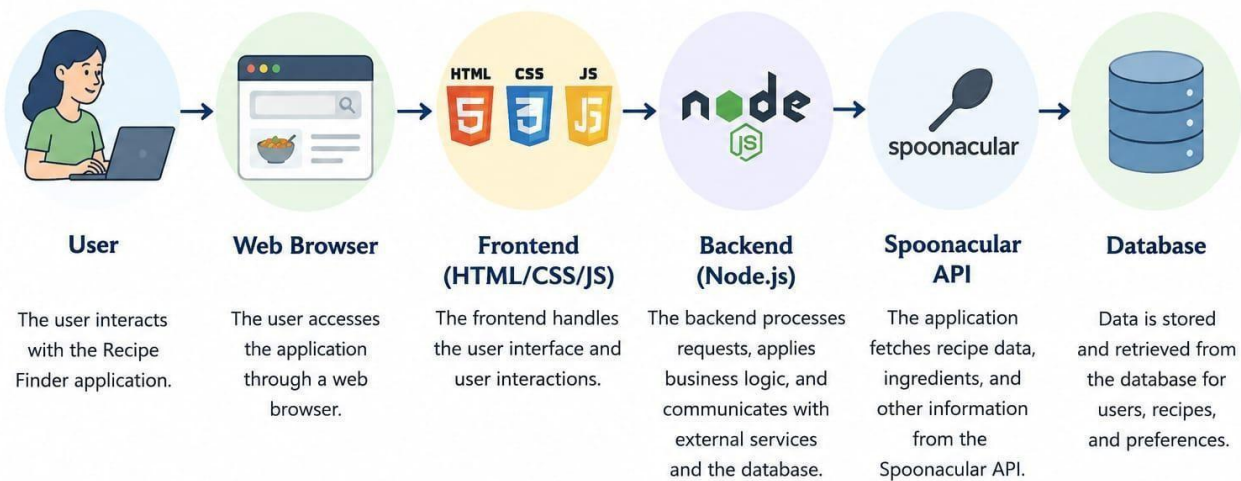
User Management Module (Module-2): This module manages user registration, login, and profile management. It stores user preferences, dietary restrictions, and bookmarked recipes in a database, enabling personalized recipe recommendations.

Recommendation Engine (Module-3): Based on user history and preferences, this module suggests recipes that the user might enjoy. It applies filtering algorithms considering the user’s dietary preferences, cuisine interests, and previously viewed recipes.

The proposed system is designed to be scalable and extensible. It can be easily adapted to new needs by adding new features or API integrations. The system is also designed to be robust and reliable, capable of handling a large number of concurrent users and recovering from network errors gracefully.

Fig 1.2: Recipe Finder Schematic Diagram

[Figure 1.2: Schematic diagram showing User → Web Browser → Frontend (HTML/CSS/JS) → Backend (Node.js) → Spoonacular API → Database]



7

Technological Environment (System Analysis & Architecture)

A recipe finder website system analysis is a process of evaluating the strengths and weaknesses of the web system in order to improve its performance and user experience. This analysis can be done by a variety of methods, including user surveys, usability testing, API performance analysis, and technical review. The goal of a recipe finder system analysis is to identify areas where the system can be improved in order to provide a better user experience. This may include improving the search algorithm, increasing the range of recipes available, optimizing API calls, or making the interface more engaging and user-friendly.

A system analysis can be a valuable tool for improving the performance of a recipe finder application. By identifying areas where the system can be improved, developers can create websites that are more useful and effective for their users.

Here are some of the key factors to consider when conducting a system analysis for the Recipe Finder Website:

- User experience: The website's ability to display relevant recipes and provide helpful, detailed cooking information is essential for a good user experience.
- Search performance: The application's ability to return accurate and relevant recipe results within an acceptable response time is crucial.
- API reliability: The dependability of the Spoonacular API and the system's ability to handle API rate limits and errors gracefully.
- Scalability: The ability of the system to handle increasing numbers of concurrent users without performance degradation.

Some of the benefits of conducting a system analysis include:

- Improved user experience: By identifying and addressing areas where the website can be improved, developers can create applications that are more user-friendly and provide a better overall experience.
- Increased efficiency: Web applications can help users to find recipes quickly, automating the search process that would otherwise require browsing multiple websites.
- Reduced costs: A well-optimized web application reduces server costs by minimizing

Study of System Modules Used:

- Search and Filter Module
- Recipe Display Module
- User Authentication Module
- Favorites and Bookmarks Module
- Nutritional Information Module
- Recommendation Engine
- Responsive UI Module

System Analysis Tools:

There are a number of system tools that can be used to create and manage recipe finder web applications. Some of the most popular tools include:

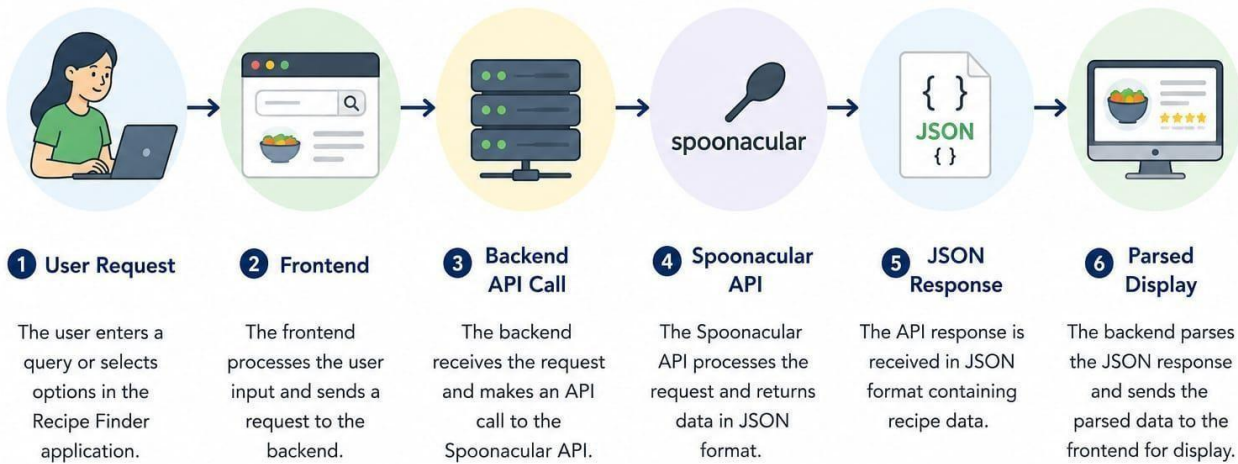
Spoonacular API: Spoonacular is a comprehensive food and recipe API platform that allows developers to build recipe finders for websites and mobile applications. Spoonacular provides endpoints for recipe search, ingredient-based lookup, nutrition analysis, meal planning, and more. The system tools used to create and manage recipe finder applications (Fig 2.1: Spoonacular API Integration). The best tool for any project will depend on specific needs and requirements.

In addition to these system tools, there are a number of other resources that can be helpful when creating and managing recipe finder websites:

- Web Development Tutorials: There are a number of tutorials available online that can teach you how to build full-stack web applications using HTML, CSS, JavaScript, and Node.js.
- Developer Communities: Online communities such as Stack Overflow, GitHub, and Reddit's r/webdev can be great resources for getting help and advice on specific technical challenges.
- API Documentation: The official Spoonacular API documentation provides comprehensive guides on available endpoints, request parameters, and response formats.

Fig 2.1: Spoonacular API Integration

[Figure 2.1: Diagram showing User Request → Frontend → Backend API Call → Spoonacular API → JSON Response → Parsed Display]



Hardware Requirements:

The hardware requirements for a Recipe Finder Website will vary depending on the scale of the deployment and the expected number of concurrent users. However, some general hardware requirements include: A web server with a powerful processor and adequate RAM to handle multiple simultaneous user requests. Sufficient storage for caching frequently accessed recipe data. A reliable and fast internet connection for API calls to Spoonacular.

Minimum Hardware Specifications:

- Processor: 2.4 GHz Dual-Core or higher
- RAM: 4 GB or higher
- Free Disk Space: 10 GB
- Internet Connection: Broadband (minimum 10 Mbps)

Software Requirements:

In addition to hardware, the following software components are required for developing and deploying the Recipe Finder Website:

- Frontend: HTML5, CSS3, JavaScript (ES6+), Bootstrap 5 for responsive design
- Backend: Node.js (v16 or higher), Express.js framework
- Database: MongoDB for storing user data, preferences, and bookmarks
- API Integration: Spoonacular API (food and recipe data)
- Version Control: Git and GitHub for source code management
- Development Tools: Visual Studio Code, Postman (API testing)
- Deployment: Heroku or Vercel for cloud deployment

PROPOSED SYSTEM:

A. Context Identification: Pre-processing is applied to the user's search input to standardize and clean the query. Based on the keywords used in the search, appropriate context is recognized – whether the user is searching by ingredient, dish name, cuisine type, or dietary restriction.

B. Ingredient-Based Search System (Module-1): Upon receiving ingredient-based queries, the system parses the ingredients list and queries the Spoonacular API with the `findByIngredients` endpoint. The results are ranked by the number of matching ingredients and displayed to the user in a card-based layout with recipe thumbnails.

C. General Recipe Search System (Module-2): If the user submits a general text query such as a dish name or cuisine type, the system uses the Spoonacular complexSearch endpoint. Results are filtered based on any applied dietary filters (vegan, vegetarian, gluten-free, etc.) and sorted by relevance or popularity.

D. User Profile and Favorites System (Module-3): Registered users can save their favorite recipes, set dietary preferences, and view their recipe search history. This data is stored in MongoDB and used to provide personalized recipe suggestions on the homepage.

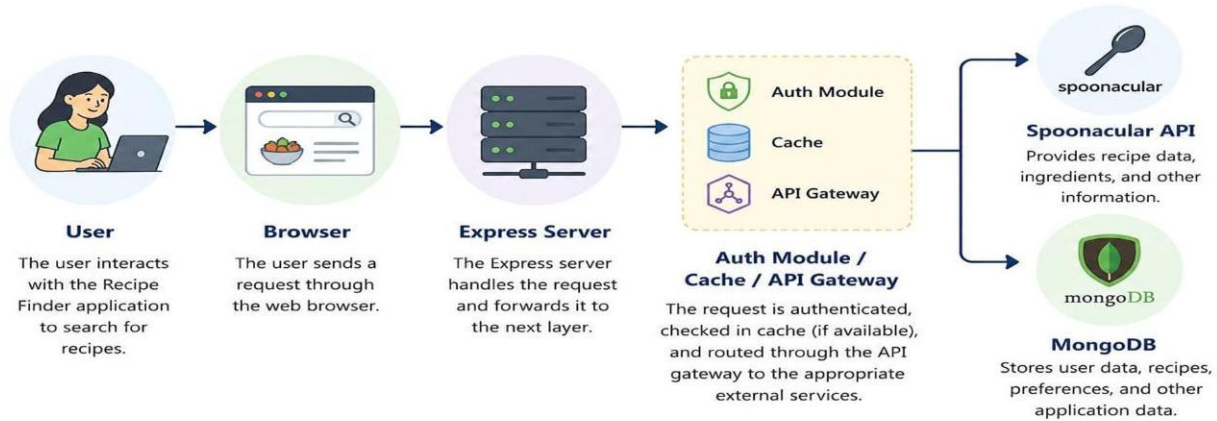
Architecture

The architecture of the Recipe Finder Website consists of several interconnected components that work together to deliver recipe information to users (Fig 2.3: Architecture of Recipe Finder Flow).

- **User Interface:** The frontend of the application built with HTML, CSS, Bootstrap, and JavaScript. This is the layer that users interact with directly.
- **Search Engine:** Processes user queries, applies filters, and calls the appropriate Spoonacular API endpoints.
- **API Gateway:** Manages API calls to the Spoonacular service, handles rate limiting, error handling, and response caching.
- **Backend Server:** A Node.js Express server that acts as middleware between the frontend and the API, handling authentication, session management, and database interactions.
- **Database:** MongoDB stores user profiles, preferences, bookmarked recipes, and search history.
- **Caching Layer:** Redis-based caching for frequently searched recipes to reduce API calls and improve response times.

Fig 2.3: Architecture of Recipe Finder Flow

[Figure 2.3: Architecture diagram showing User → Browser → Express Server → Auth Module / Cache / API Gateway → Spoonacular API and MongoDB]



Advantages:

The Recipe Finder Website offers a number of advantages over traditional recipe searching methods such as cookbooks or unstructured web searches.

- **Instant Search:** Users receive recipe results in real-time as they type, significantly reducing the time spent searching for recipes.
- **Ingredient-Based Search:** A unique feature that allows users to input the ingredients they already have and find recipes that use them, reducing food waste.
- **Dietary Filtering:** Comprehensive filters for dietary restrictions (vegan, vegetarian, keto, gluten-free, etc.) make meal planning easy for individuals with specific dietary needs.
- **Nutritional Transparency:** Every recipe comes with detailed nutritional information including calories, protein, carbohydrates, fats, vitamins, and mine

8

FEASIBILITY REPORT

Preliminary investigation examines project feasibility, the likelihood the system will be useful to the organization. The main objective of the feasibility study is to test the Technical, Operational, and Economical feasibility for adding new modules and debugging the running system.

All systems are feasible if they are given unlimited resources and infinite time. There are three aspects in the feasibility study portion of the preliminary investigation:

- Technical Feasibility
- Operational Feasibility
- Economical Feasibility

TECHNICAL FEASIBILITY:

The technical issue usually addressed during the feasibility stage of the investigation includes the following:

- Does the necessary technology exist to do what is suggested? Yes – HTML, CSS, JavaScript, Node.js, and the Spoonacular API are all mature, well-documented technologies.
- Do the proposed technologies have the technical capacity to handle the data required to use the new system? Yes – MongoDB is a highly scalable NoSQL database capable of handling large volumes of recipe and user data.
- Will the proposed system provide adequate response to user queries, regardless of the number or location of users? Yes – cloud deployment on platforms like Heroku or Vercel ensures global accessibility and scalability.
- Can the system be upgraded if developed? Yes – the modular architecture allows for easy addition of new features and API integrations.
- Are there technical guarantees of accuracy, reliability, ease of access, and data security? Yes – HTTPS encryption, JWT-based authentication, and API key security ensure data protection.

The current system developed is technically feasible. It is a web-based user interface that provides an easy access to users from any device with a web browser. The database's purpose is to create, establish and maintain user preferences and recipe data. Permission to users would be granted based on their registration and authentication status.

OPERATIONAL FEASIBILITY

Proposed projects are beneficial only if they can be turned out into information systems that will meet the organization's operating requirements. Operational feasibility aspects of the project are to be taken as an important part of the project implementation.

Some of the important issues raised to test the operational feasibility of a project include the following:

- Is there sufficient support for the system from users? Yes – the demand for online recipe discovery continues to grow, with millions of users searching for recipes daily.
- Will the system be used and work properly if it is developed and implemented? Yes – extensive testing including unit testing, integration testing, and user acceptance testing will ensure reliable operation.
- Will there be any resistance from the users that will undermine the possible application benefits? Minimal – the intuitive and familiar interface design minimizes the learning curve for new users.

This system is targeted to be in accordance with the above-mentioned issues. The well-planned design would ensure the optimal utilization of system resources and would help in the improvement of performance status.

ECONOMICAL FEASIBILITY

- A system can be developed technically and will be used if installed must still be a good investment. In the economic feasibility, the development cost in creating the system is evaluated against the ultimate benefit derived from the new system. Financial benefits must equal or exceed the cost.
- The system is economically feasible. Development uses open-source technologies (Node.js, MongoDB, Bootstrap) which are freely available. The Spoonacular API offers a free tier suitable for development and limited production use, with affordable paid plans for larger scale.
- The interface for this system is developed using freely available resources, minimizing development costs. Deployment costs on cloud platforms are minimal for small to medium-scale applications.

SOFTWARE REQUIREMENT SPECIFICATION

To create a software requirements specification (SRS) for the Recipe Finder Website, the functional and non-functional requirements of the system must be defined. Here are some key elements to consider when documenting the SRS:

Introduction

This document provides an overview of the Recipe Finder Website project. It defines the purpose and goals of the web application and specifies the target audience or user base. The Recipe Finder Website aims to provide an intuitive, fast, and comprehensive platform for users to discover recipes based on their ingredients, dietary preferences, and cuisine interests.

1 User Interface Requirements:

1. Describe the user interface elements of the website, including text-based search inputs, filter dropdowns, recipe cards, and detailed recipe view pages.
2. Specify requirements related to the website's appearance, including a clean and modern design with responsive layout for all screen sizes.

2 API Integration Requirements:

3. Define the APIs the website should integrate with – primarily the Spoonacular API for recipe data.
4. Specify any specific API capabilities required, such as ingredient-based search, complex recipe search, nutritional information retrieval, and random recipe generation.
5. Outline requirements related to API error handling, rate limit management, and response caching.

3 Functional Requirements:

- The website must allow users to search for recipes by entering a dish name, ingredient, or cuisine type in a search bar.
- The website must provide filter options including diet type (vegan, vegetarian, gluten-free, ketogenic, etc.), cuisine type (Italian, Indian, Chinese, Mexican, etc.), and cooking time.
- The website must display search results as recipe cards showing the recipe image, name,

preparation time, and calorie count

The website must provide a detailed recipe view showing full ingredient list with quantities, step-by-step cooking instructions, servings, preparation time, and nutritional breakdown .

- The website must allow registered users to bookmark/save their favorite recipes for future reference.
- The website must support user registration and login with secure password hashing.
- The website must provide a personalized homepage for logged-in users showing recommended recipes based on their preferences.
- The website must display nutritional information including calories, protein, carbohydrates, fats, fiber, vitamins, and minerals for each recipe.
- The website must integrate with the Spoonacular API to access a database of

4 Performance Requirements:

Response Time:

The website must be able to respond to user search queries within 2-3 seconds under normal network conditions. The response time depends on factors including the complexity of the search query, current API load, and network latency. In general, the system should return recipe results within 2 seconds for simple searches and within 5 seconds for complex nutritional queries.

Accuracy:

The search algorithm must return relevant recipes with an accuracy rate of at least 90% for exact recipe name matches and at least 75% for ingredient-based searches. Accuracy is an important metric as it directly impacts user satisfaction. Factors affecting accuracy include the quality of the Spoonacular recipe database, the precision of the search algorithm, and the relevance ranking system.

User Engagement:

The website must be designed to maximize user engagement through an intuitive and visually appealing interface. Engagement metrics include time spent on the site, number of recipes viewed per session, and the recipe save/bookmark rate. Users are more likely to engage with a website that presents information clearly and offers relevant suggestions.

Scalability:

The website must be able to scale to meet the needs of a growing user base. The system should be able to handle at least 1,000 concurrent users without significant performance degradation. Scalability is achieved through the use of cloud hosting, database indexing, API response caching, and a stateless RESTful backend architecture. Cloud deployment on platforms like Heroku allows automatic scaling based on traffic demands.

Chatbot Process Flow Diagram:

Wondering what the Recipe Finder process flow is all about? While recipe finder websites seem straightforward in terms of helping users find meals, the underlying data flow is more complex (Fig 4.2: Recipe Finder process flow diagram 1 and diagram 2).

Fig 4.2: Recipe Finder Process Flow Diagram-1

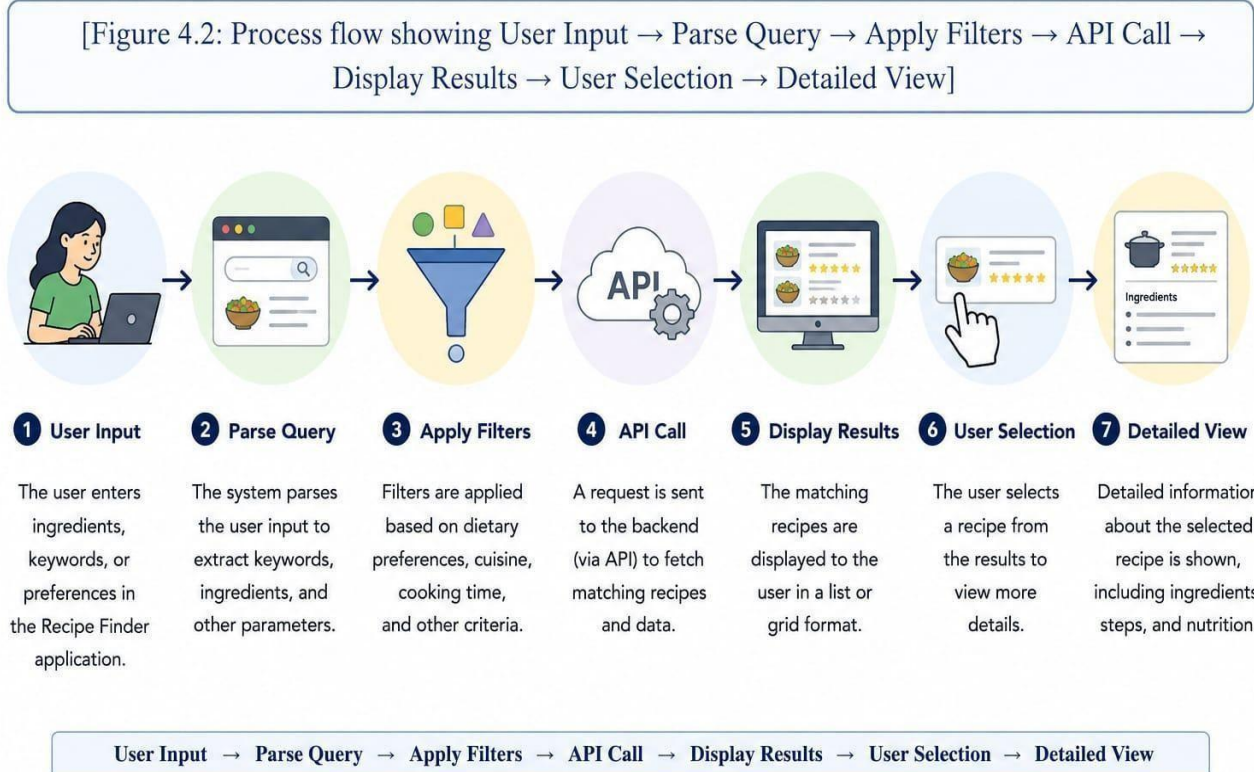
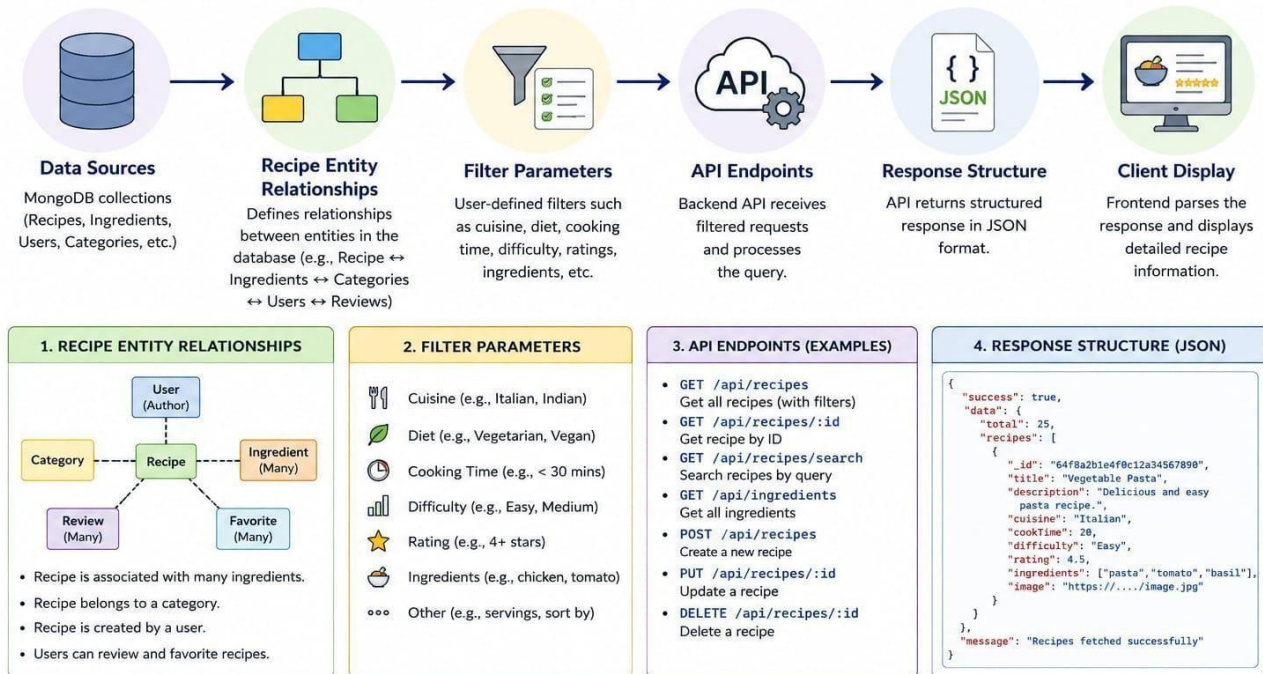


Fig 4.2: Recipe Finder Process Flow Diagram-2

[Figure 4.2: Detailed data flow showing Recipe Entity relationships, Filter parameters, API endpoints, and Response structure]



The seamless flow of information from user input to recipe display constitutes the core workflow of the Recipe Finder Website. A natural search experience, guided by intelligent filtering and relevant results, provides a pleasant and efficient cooking discovery experience. Designing a well-structured information flow is a crucial component of the development process for a recipe finder website.

People are accustomed to digital recipe platforms like BBC Good Food and Allrecipes that provide sophisticated filtering and detailed recipe information. Consumer research suggests that users expect recipe websites to have comprehensive search capabilities, clear nutritional information, and responsive design across devices.

INTRODUCTION TO SPOONACULAR API & FRONTEND FRAMEWORK

The Spoonacular API is a comprehensive food and recipe data platform that makes it easy to integrate recipe search, nutritional analysis, meal planning, and ingredient management into web and mobile applications. The Spoonacular API can process multiple types of search inputs from users, including ingredient lists, dish names, cuisine preferences, and dietary restriction filters. It can respond with detailed recipe data including JSON-formatted ingredient lists, step-by-step instructions, nutritional information, and high-quality recipe images.

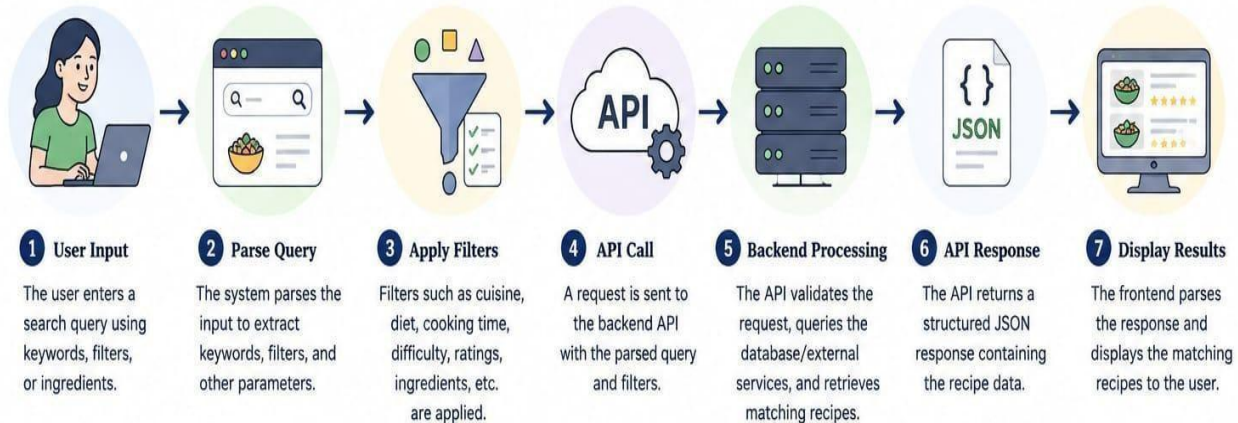
Spoonacular is a powerful tool that can be used to create a variety of recipe-based experiences, including:

- Recipe finders that allow users to search by ingredients, dish name, or nutritional requirements.
- Meal planners that create weekly meal plans based on caloric goals and dietary preferences.
- Ingredient substitution tools that suggest alternatives for unavailable ingredients.
- Nutritional analyzers that calculate the nutritional content of custom recipes.

To get started with the Spoonacular API, developers need to create an account on RapidAPI or Spoonacular's developer portal and obtain an API key. The key endpoints used in this project include the complex recipe search endpoint, the find recipes by ingredients endpoint, the get recipe information endpoint, and the get recipe nutrition endpoint (Fig 5.1: Life of a Search Query).

Fig 5.1: Life of a Search Query

[Figure 5.1: Diagram showing the complete lifecycle of a search query from user input through API processing to recipe display]



Key API Endpoints Used in This Project

GET /recipes/complexSearch

Search recipes with multiple filters (cuisine, diet, time, etc.)

GET /recipes/findByIngredients

Find recipes by a list of ingredients

GET /recipes/{id}/information

Get detailed information about a specific recipe

GET /recipes/{id}/nutritionWidget

Get nutrition information for a specific recipe

Example Response Structure (JSON)

```
{
  "success": true,
  "data": {
    "total": 25,
    "recipes": [
      {
        "id": "64f8a2b1e4f0c12a34567890",
        "title": "Vegetable Pasta",
        "image": "https://.../image.jpg",
        "cuisines": "Italian",
        "readyInMinutes": 20,
        "servings": 4,
        "ingredients": ["pasta", "tomato", "basil"],
        "healthScore": 85,
        "spoonacularScore": 92.5
      }
    ]
  },
  "message": "Recipes fetched successfully"
}
```

- **success:** Indicates whether the request was successful.
- **data:** Contains the result payload.
- **total:** Total number of recipes found.
- **recipes:** Array of recipe objects.
- Each recipe object includes details such as:
 - **id:** Unique recipe ID
 - **title:** Recipe title
 - **image:** Image URL
 - **cuisines:** Cuisine type
 - **readyInMinutes:** Cooking time
 - **servings:** Number of servings
 - **ingredients:** List of ingredients
 - **healthScore / spoonacularScore:** Recipe scores
- **message:** Status message from the API.

SPOONACULAR API FEATURES

Spoonacular is a natural language food understanding platform that makes it easy to design and integrate recipe and nutrition data into web applications. It has a number of features that make it a powerful tool for creating recipe-based experiences, including:

- **Complex Recipe Search:** Allows searching by query, cuisine, diet, intolerances, ingredients, and meal type simultaneously. Supports over 50 filter parameters.
- **Find by Ingredients:** Given a comma-separated list of ingredients, returns recipes that use these ingredients. Results can be ranked by ingredient match count or missing ingredients.
- **Recipe Information:** Returns comprehensive details about a specific recipe including servings, preparation time, ingredients with amounts, step-by-step instructions, and nutritional information.
- **Nutrition Widget:** Provides a detailed nutritional breakdown for any recipe, including macros (protein, carbs, fats) and micronutrients (vitamins, minerals).
- **Random Recipes:** Returns a random set of recipes, useful for homepage features and daily recipe suggestions.
- **Ingredient Information:** Provides details about specific ingredients including nutritional content, possible substitutions, and conversion units.
- **Meal Planning:** Generates complete meal plans for a day or week based on caloric goals and dietary preferences.

Here are some of the key features of the Spoonacular API framework:

- **RESTful Architecture:** Spoonacular uses a REST API that returns JSON responses, making it easy to integrate with any frontend or backend technology.
- **Comprehensive Database:** Access to over 360,000 recipes from various sources, covering virtually every cuisine and dietary preference.
- **Powerful Filtering:** Advanced filtering capabilities including multiple simultaneous filters for diet, cuisine, ingredients, and nutritional constraints.
- **Nutritional Analysis:** Detailed nutritional data for all recipes, supporting health-conscious meal planning.
- **Affordable Pricing:** A free tier is available for development and testing, with paid plans for production use.

Frontend Architecture:

The frontend architecture of the Recipe Finder Website is a responsive single-page application (SPA) approach that consists of the following components:

HTML Layer:

The semantic HTML5 structure defines the layout of the website including the navigation bar, search section, filter panel, recipe grid, recipe detail modal, and user profile section.

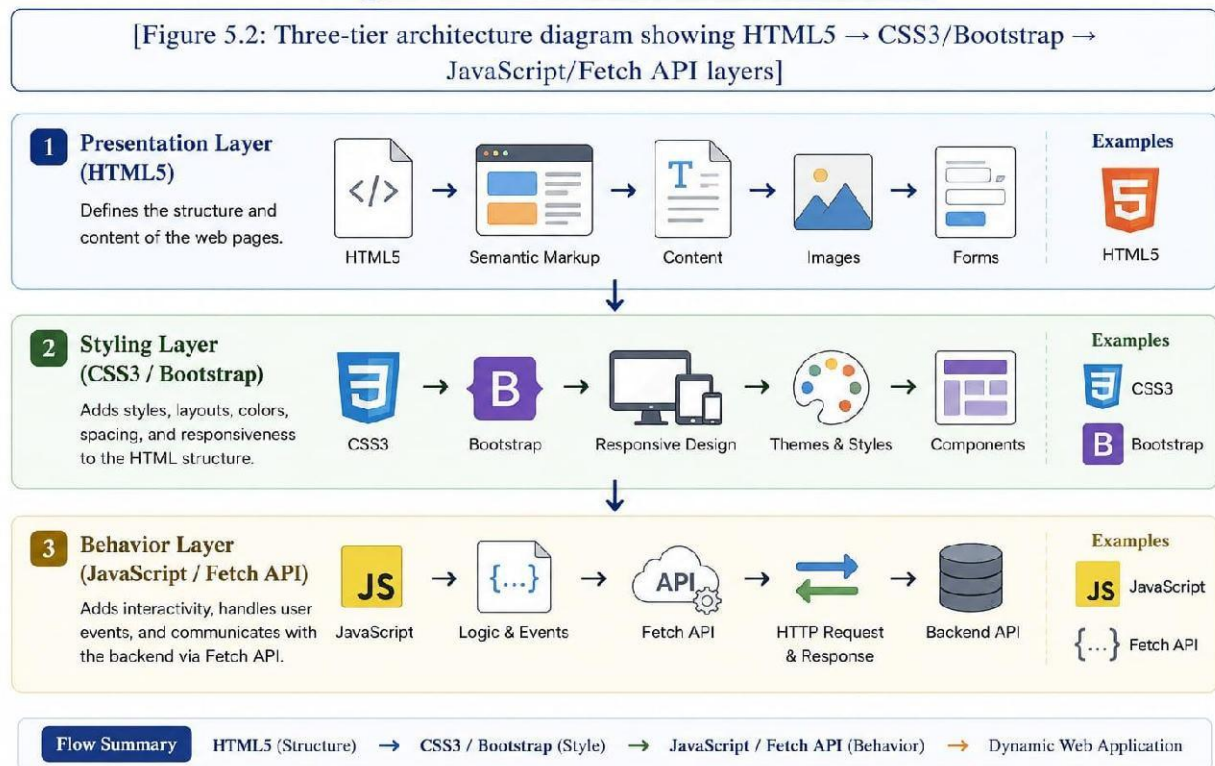
CSS Layer:

Bootstrap 5 provides the responsive grid system and pre-built UI components. Custom CSS handles the unique visual identity of the website including color schemes, typography, card hover effects, and loading animations.

JavaScript Layer:

Vanilla JavaScript handles DOM manipulation, API calls using the Fetch API, user input handling, filter logic, local storage for temporary data, and dynamic content rendering.

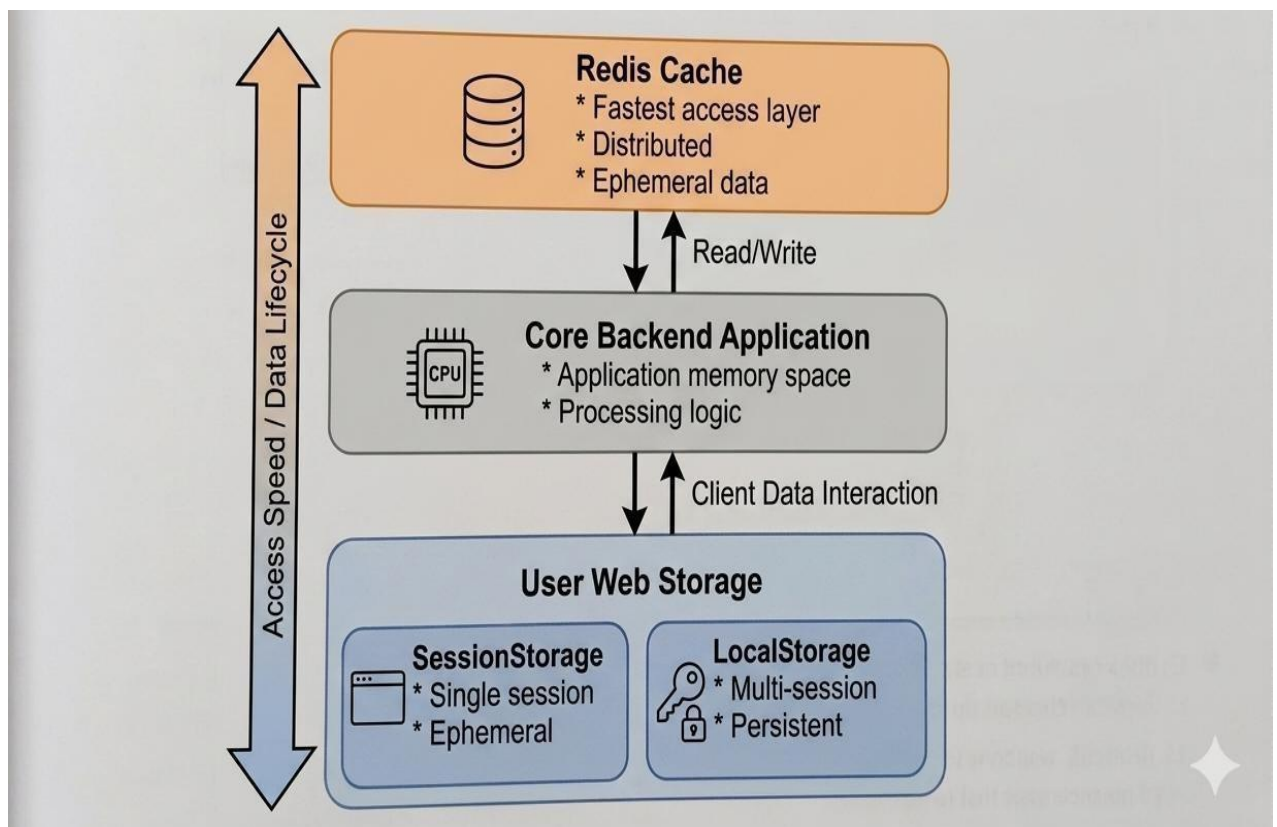
Fig 5.2: Frontend Architecture



MEMORY MANAGEMENT:

The Recipe Finder Website uses a variety of techniques to manage data efficiently (Fig 5.3: Memory Management). Some of these techniques include:

- **Browser Caching:** The browser's built-in caching mechanism stores recently fetched recipe data, reducing redundant API calls when the user revisits previously searched recipes.
- **LocalStorage:** User preferences, dietary filters, and recently viewed recipes are stored in the browser's LocalStorage for a persistent user experience across sessions.
- **SessionStorage:** Temporary search results and pagination state are stored in SessionStorage to maintain context during the current browsing session.
- **API Response Caching:** The backend server implements a Redis-based caching layer that stores API responses for popular searches, significantly reducing API call volume and improving response times.
- **Lazy Loading:** Recipe images are loaded lazily, meaning they are only fetched when they enter the viewport. This significantly reduces initial page load time.
- **Pagination:** Search results are paginated to limit the amount of data loaded at once, improving performance and user experience.



Flexible Architecture of the Recipe Finder Website:

- The Recipe Finder Website follows a modular, flexible architecture that allows for extensibility and customization. Here's a high-level overview:
- **User Interface Layer:** Developers can define and modify the UI components, search flow, and recipe display layouts. The modular CSS and JavaScript structure makes it easy to update individual components without affecting the rest of the application.
- **API Integration Layer:** The API service module abstracts all Spoonacular API calls, making it easy to add new endpoints or switch to a different recipe API if needed.
- **State Management:** The application uses a lightweight state management approach with JavaScript objects and LocalStorage to maintain application state without requiring a heavy framework.
- **Authentication Module:** JWT-based authentication is implemented as a separate module that can be easily updated or replaced with OAuth-based authentication in the future.
- **Analytics and Monitoring:** Integration with Google Analytics allows tracking of user interactions, popular search terms, and recipe engagement metrics, providing data-driven insights for future development.

11

SYSTEM DESIGN

The Recipe Finder Website is a web-based application that provides users with a comprehensive platform for recipe discovery, search, and detailed viewing. The system design for such a web application typically includes the following components:

- **Frontend Module (HTML/CSS/JS):** The user-facing layer responsible for rendering recipe search results, filter options, and detailed recipe views in a visually appealing and responsive interface.
- **Backend Server (Node.js/Express):** The server-side layer responsible for handling API requests from the frontend, managing user authentication, and serving data from the database and third-party APIs.
- **Database (MongoDB):** Stores persistent data including user profiles, saved preferences, bookmarked recipes, and search history.
- **API Integration Module:** Manages communication with the Spoonacular API, handles API key management, error handling, and response parsing.

The system design for the Recipe Finder Website is carefully structured to ensure separation of concerns, scalability, and maintainability. A well-designed web application can provide a valuable service to users, while a poorly-designed one can be frustrating and lead to high bounce rates.

Design Technique / Procedure:

Any designer generally follows five necessary steps before designing a Recipe Finder Website. The first step is to determine the Website's purpose (What do users need from a recipe finder?). After that, the designer must decide between a keyword-based search approach or an ingredient-based AI matching system.

The five steps are as follows (Fig 6: Designing a Recipe Finder Website):

- **Step 1: Determine the Purpose of the Website** – Define whether the primary use case is general recipe search, ingredient-based meal planning, or nutrition tracking.
- **Step 2: Decide Between Simple Search and Advanced Filtering** – Determine whether to offer a simple search bar or an advanced multi-parameter search with filters.

Step 3: Design the User Interface Like a User Would Browse – Design the layout as if browsing a digital cookbook, with visual recipe cards, clear typography, and intuitive navigation

Step 4: Design the Conversation/Search Flow – Map out the complete user journey from landing on the homepage to finding and saving a recipe.

Step 5: Select a Suitable Platform for Deployment – Choose a hosting platform such as Vercel, Heroku, or Netlify that ensures global accessibility and performance.

Fig 6: Designing a Recipe Finder Website

Figure 6: Five-step design flow: Determine Purpose → Choose Architecture → Design UI → Flow → Select Deployment Platform]



Normalization:

Normalization in the Recipe Finder Website context refers to the standardization of user search inputs and recipe data. This includes removing special characters, standardizing ingredient names (e.g., 'tomato' vs 'tomatoes'), converting units, and normalizing cuisine names. Normalization improves the accuracy of recipe matching and ensures consistent data storage.

Some normalization techniques used in this system:

- **Input Sanitization:** User search inputs are sanitized to remove potentially harmful characters and standardized to lowercase for consistent API querying.
- **Ingredient Normalization:** Common ingredient name variations are mapped to standard names (e.g., 'AP flour', 'plain flour' all map to 'all-purpose flour') to improve search accuracy.
- **Unit Conversion:** Recipe quantities are normalized to standard metric or imperial units based on user preferences.
- **Calorie Normalization:** Nutritional data is normalized per serving to allow consistent comparison between recipes of different yields.

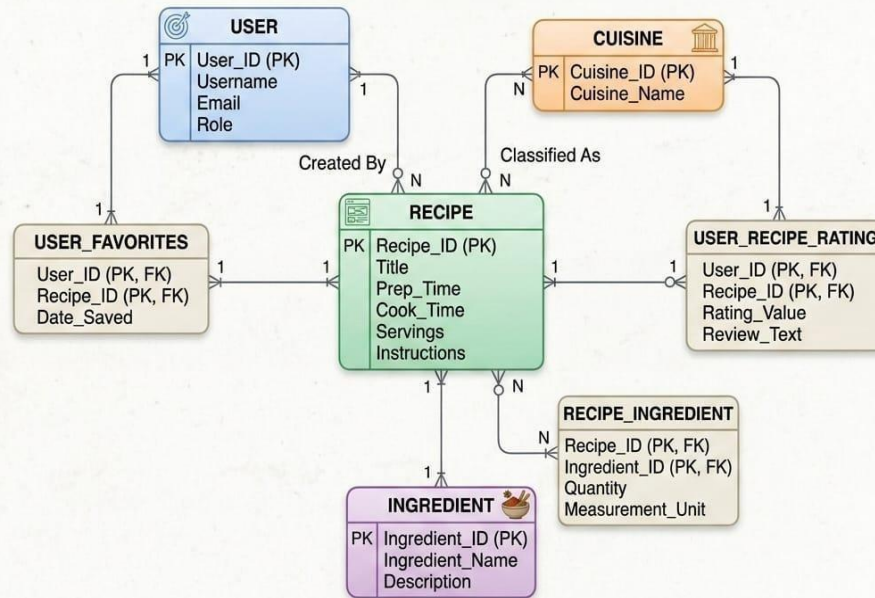
Entity Relationship (ER Diagram)

The Entity Relationship (ER) Diagram for the Recipe Finder Website describes the relationships between the main data entities in the system. The key entities and their relationships are as follows:

- **User Entity:** Attributes include UserID (Primary Key), Username, Email, Password (hashed), Created Date, Dietary Preferences, and Cuisine Preferences.
- **Recipe Entity:** Attributes include RecipeID, Title, Image URL, Preparation Time, Servings, Cuisine Type, Diet Type, and Source URL.

Fig 6: Designing a Recipe Finder Website

Figure 7: Data Model for Recipe Finder Website (Conceptual ER Diagram)



Relationships: A User can save many Recipes (Many-to-Many through Saved Recipe). A Recipe has one Nutritional Info record (One-to-One). A Recipe has many Ingredients (One-to-Many).

Data Flow Diagrams (DFDs)

A data flow diagram (DFD) is a graphical representation of the flow of data through a system. DFDs can be used to model the Recipe Finder Website's data flow at different levels of abstraction (Fig 6.1: level-0 and level-1 of DFD).

A DFD for the Recipe Finder Website would typically include the following components:

- **External Entities:** External entities represent the users of the website and the Spoonacular API.
- **Processes:** Processes represent the steps the system takes to process user input and generate recipe results.
- **Data Stores:** Data stores represent the MongoDB database and browser storage.

6.1 DFD COMPONENTS AND LEVEL-0 DIAGRAM

A DFD for the Recipe Finder Website would typically include the following components:

- **External Entities:** Represent the users of the website of the website and the Spoonacular API.
- **Processes:** Popresses represent the steps the system takes to process user input and generate recipe results.
- **Data Stores:** Represent represent the MongoDB database and browser storage.

▲ **Fig 6.1: Level-0 of DFD**

[Figure 6.1 Level-0: User → [Search Query] → Recipe Finder System → [Recipe Results] → User; Admin → [Update Config] → System]

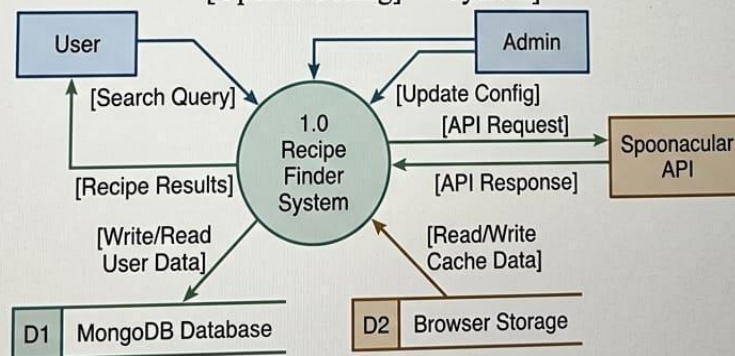


Fig 6.1: Level-1 of DFD

[Figure 6.1 Level-1: Expanded DFD showing Login -> Search -> Filter -> API Call -> Cache Check -> Display Results -> Save Favorite sub-processes]

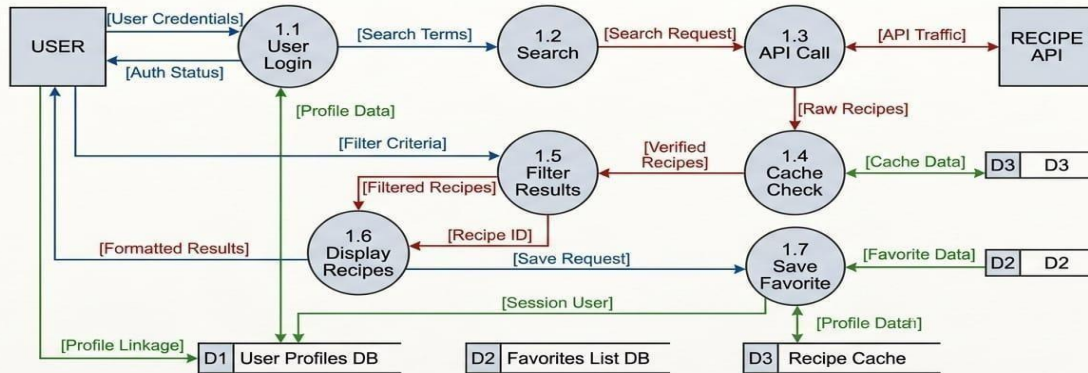
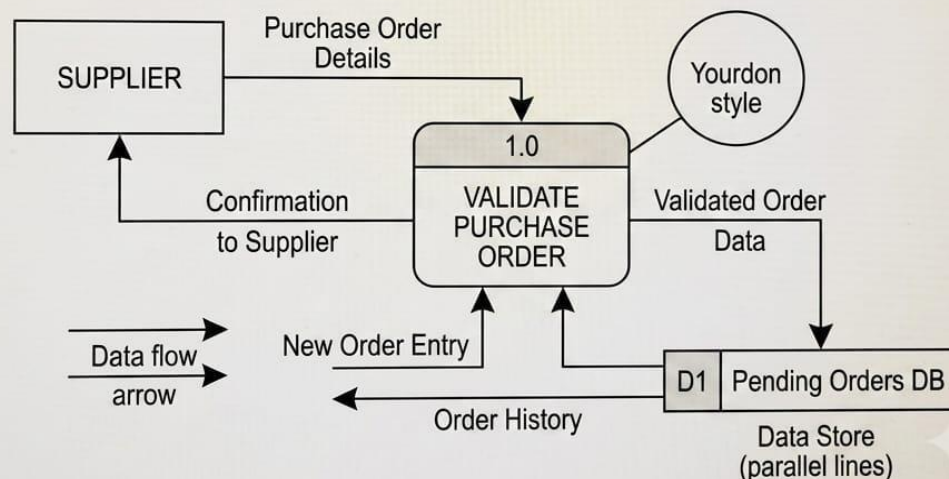


Fig 6.1.1: Dataflow Diagram

[Figure 6.1.1: DFD notation reference showing External Entity (rectangle), Process (circle/rounded rectangle), Data Store (parallel lines), Data Flow (arrow)]

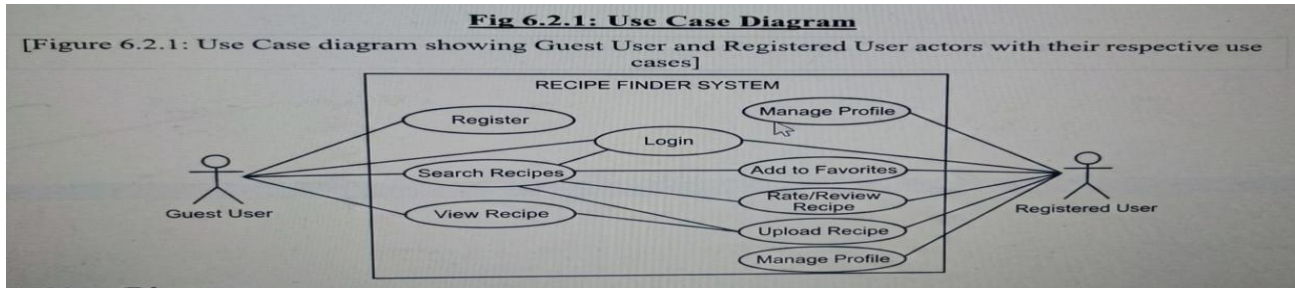


Unified Modeling Language (UML) is a visual modeling language that is used to describe the structure and behavior of software systems. UML can be used to model a wide variety of systems, including web-based applications like the Recipe Finder Website.

The following are some of the most common UML diagrams used to model the Recipe Finder

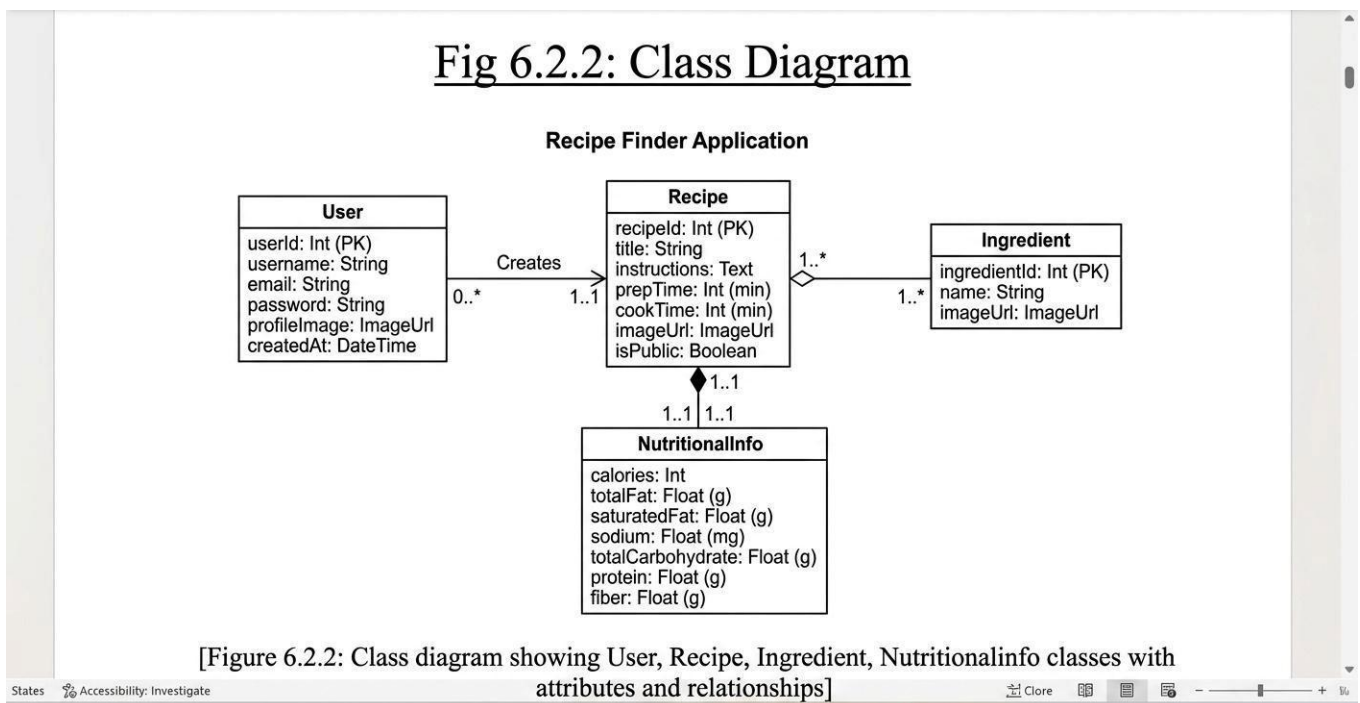
Use Case Diagram:

A use case diagram shows the different ways that users can interact with the Recipe Finder Website. Key use cases include: Search Recipes, Filter by Diet/Cuisine, View Recipe Details, Save Favorite Recipe, Register/Login, View Nutritional Info, and Get Random Recipe Suggestion (Fig 6.2.1: Use Case Diagram).



Class Diagram:

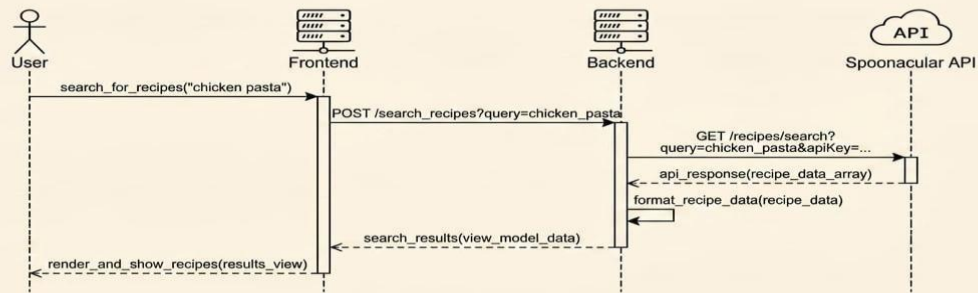
A class diagram shows the different classes of objects in the system and the relationships between those classes. Key classes include: User, Recipe, Ingredient, NutritionalInfo, SearchQuery, FilterOptions, and APIService (Fig 6.2.2: Class Diagram).



Sequence Diagram:

A sequence diagram shows the order in which messages are exchanged between objects in the system. It models the interaction flow from a user's search query to the display of recipe results (Fig 6.2.3: Sequence Diagram).

Figure 6.2.3: Sequence diagram showing User → Frontend → Backend → Spoonacular API



[Figure 6.2.3: Sequence diagram showing User → Frontend → Backend → Spoonacular API message with return responses]

Activity Diagram:

An activity diagram shows the flow of control in the system. It models the different steps a user takes from landing on the website to finding, viewing, and saving a recipe (Fig 6.2.4: Activity Diagram).

Fig 6.2.4: Activity Diagram

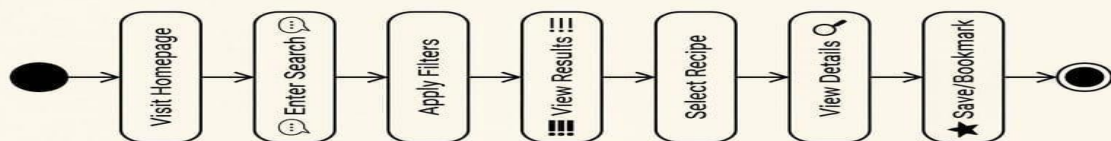


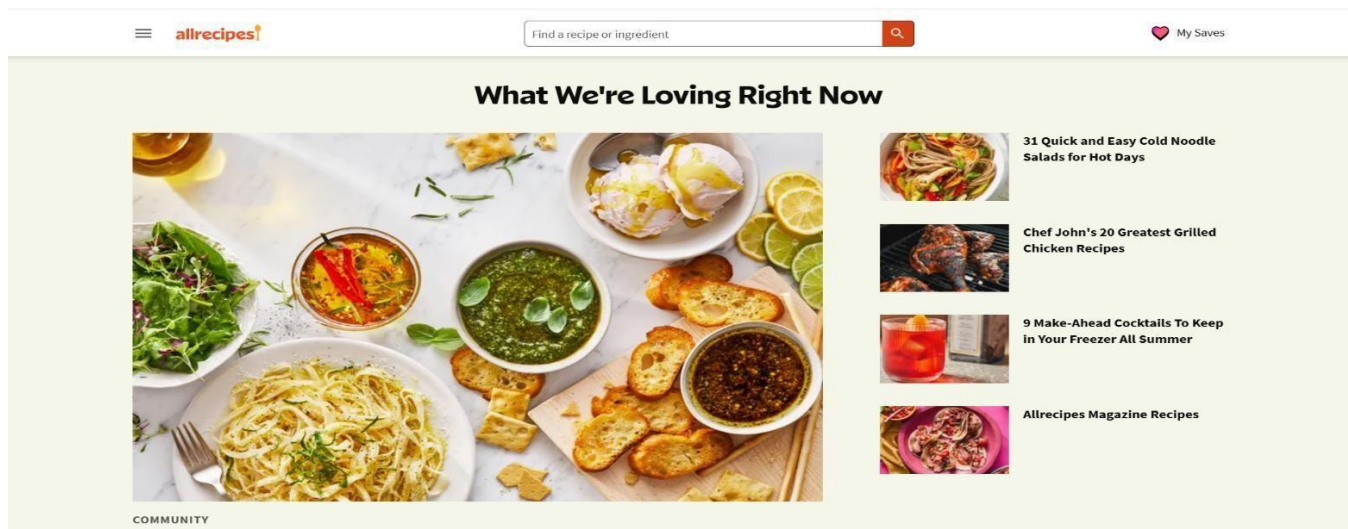
Figure 6.2.4: Activity diagram showing Start → Visit Homepage → Enter Search → Apply Filters → View Results → Select Recipe → View Details → Save/Bookmark → End

12

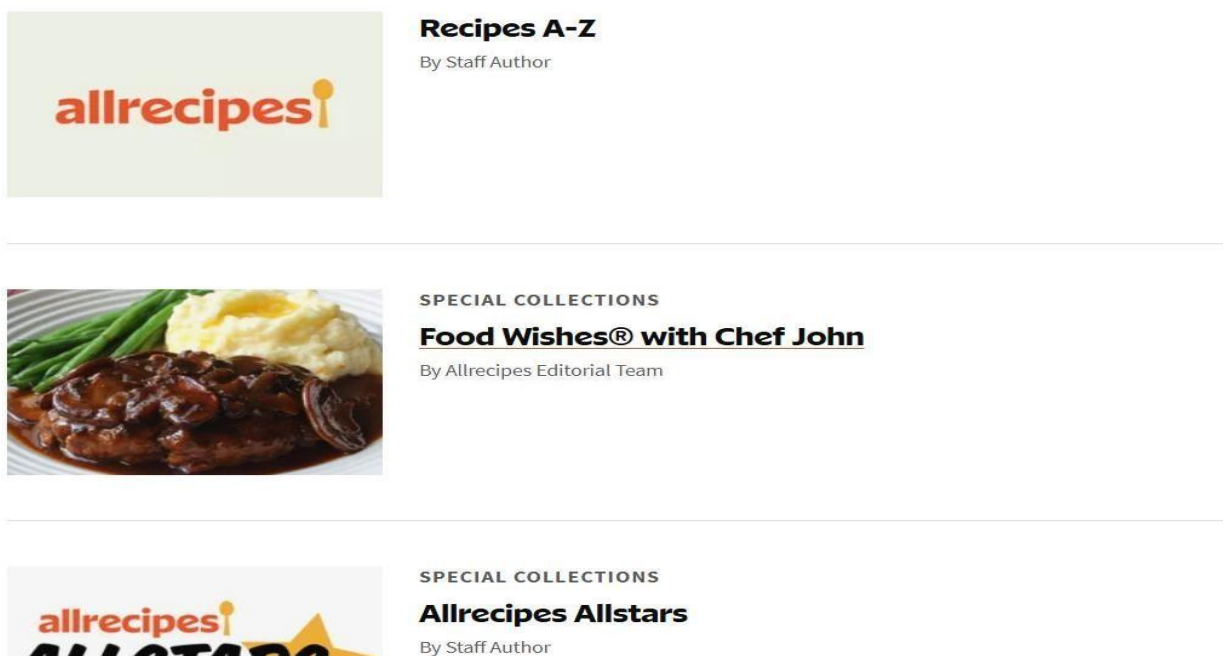
OUTPUT SCREENS

The following screenshots demonstrate the key screens and functionalities of the Recipe Finder Website as observed during testing and demonstration.

Output Screen 1: Homepage with Search Bar



Output Screen 2: Recipe Search Results Page



SYSTEM TESTING AND IMPLEMENTATION

Some of the key considerations for system testing for the Recipe Finder Website:

- **Functional requirements:** The website must meet all functional requirements including search, filter, recipe display, user authentication, and bookmarking.
- **Performance requirements:** The system must handle a large number of concurrent users and respond to search queries within the specified time limits.
- **Usability requirements:** The website must be easy to navigate and use across different devices and browsers.
- **API reliability:** The integration with the Spoonacular API must be tested for correct data retrieval and error handling.

INTRODUCTION:

System testing for the Recipe Finder Website is the process of evaluating the website's functionality, performance, and usability. This type of testing is typically performed after the website has been developed and before it is deployed to production. There are a number of different ways to test web applications. Testing is the process of finding differences between the expected behavior defined by system requirements and the observed behavior of the implemented system.

From a quality assurance perspective, testing is the attempt to find defects in the system with respect to the system specifications. The goal of testing is to design tests that reveal errors and demonstrate that the system behaves as expected. The process of executing a web application with a set of test cases and evaluating the output is called testing.

A Strategic Approach to Software Testing:

A strategic approach to software testing is a plan that outlines the goals, objectives, and methods of software testing. A strategic approach helps ensure that the web application is tested effectively and efficiently. The strategic approach includes:

- **Goals and Objectives:** Ensure the Recipe Finder Website meets all functional and non-functional requirements as specified in the SRS document.
- **Methods:** Unit testing, integration testing, system testing, acceptance testing, and cross-browser compatibility testing.

- Resources: Development team, QA engineers, automated testing tools (Jest, Selenium), and real user feedback.
- Schedule: Testing phases aligned with development sprints, with final regression testing before deployment.
- Communication: Regular testing reports shared with the development team and project supervisor.

Unit Testing:

Unit testing focuses on individual components of the Recipe Finder Website in isolation. Each JavaScript function, API service module, and UI component is tested independently to verify its correct behavior (Fig 8.2.1: Unit Testing).

Unit testing reduces the overall testing complexity by focusing on the smallest units of the system. It makes it easier to pinpoint and correct defects since few components are involved. Unit testing also allows parallel testing, where multiple team members test different components simultaneously.

Examples of unit tests for the Recipe Finder Website:

- Test that the search function correctly constructs API query parameters from user input.
- Test that the filter module correctly applies dietary and cuisine filters to search results.
- Test that the recipe card component correctly renders recipe data from the API response.
- Test that the user authentication module correctly validates and stores JWT tokens.
- Test that the bookmark module correctly saves and removes recipes from the user's favorites list

Integration Testing:

In integration testing, multiple modules are combined into subsystems and tested together. The goal is to verify that the modules can be integrated correctly and that their interactions produce the expected results. After individual module testing, integration testing verifies that the frontend correctly communicates with the backend, the backend correctly queries the Spoonacular API, and the database correctly stores and retrieves user data

Examples of integration tests:

- Test that a user search query on the frontend triggers a correct API call on the backend and returns properly formatted recipe results.
- Test that a user's saved recipe on the frontend is correctly persisted in the MongoDB database.
- Test that the authentication flow correctly validates user credentials and returns a JWT token that grants access to protected routes.

System Testing:

System testing evaluates the entire Recipe Finder Website as a complete integrated system. The reference for this process is the SRS document, and the goal is to verify that the system meets all specified requirements. The website is tested with various test cases including different search queries, filter combinations, user interactions, and edge cases such as empty search results, API timeout handling, and invalid user input.

Acceptance Testing:

Acceptance testing is performed by end users to ensure that the Recipe Finder Website meets their needs and expectations. This is the last phase of testing before deployment. User Acceptance Testing (UAT) involves real users performing typical tasks such as searching for recipes, applying filters, viewing recipe details, and saving favorites. Feedback from UAT is used to make final adjustments to the UI and functionality before launch.

TESTING TECHNIQUES:

- Black Box Testing (Functional Testing)
- White Box Testing (Structural Testing)

Black Box Testing:

This method is used when the internal implementation is not considered – only the inputs and expected outputs. In black box testing for the Recipe Finder Website, testers interact with the website as a regular user would, without knowledge of the internal code structure (Fig 8.3.1: Black Box Testing).

Black box testing attempts to find errors in the following categories:

- Incorrect or missing search results for valid search queries.
- Interface errors – broken links, missing images, non-responsive elements.
- Filter errors – dietary filters not correctly applied to results.
- Performance errors – slow response times exceeding the 5-second threshold.
- Authentication errors – incorrect login/logout behavior.

White Box Testing:

White box testing examines the internal structure and logic of the application. In the Recipe Finder Website, white box testing involves reviewing the JavaScript functions, API call logic, error handling code, and database query logic to ensure they are implemented correctly and efficiently (Fig 8.3.2: White Box Testing).

White box testing is also known as structural testing or code-based testing. Testers with access to the source code design test cases that verify the correctness of the implementation at the code level. This includes path coverage testing, branch testing, and statement coverage analysis

IMPLEMENTATION:

A. Search Module Implementation: The core search functionality is implemented using JavaScript's Fetch API to make asynchronous calls to the backend server, which then queries the Spoonacular API. The search module handles query construction, response parsing, and error management. Results are rendered dynamically into the recipe grid using DOM manipulation.

B. Ingredient Normalization: Input ingredients entered by users are normalized by converting to lowercase, trimming whitespace, removing special characters, and splitting comma-separated values into an array. This array is then passed as a parameter to the Spoonacular `findByIngredients` endpoint.

C. Local Storage Management: User preferences (preferred cuisine, dietary restrictions) and recent searches are stored in the browser's LocalStorage. This data is loaded on page initialization to provide a personalized experience from the first interaction.

D. Recipe Similarity and Recommendation: For the recommendation feature, similarity is computed based on the user's saved recipe tags (cuisine type, diet type, main ingredient). Recipes with the highest tag overlap with the user's saved collection are prioritized in recommendation

Conclusion and Future Scope

The development of the Recipe Finder Website marks a significant step toward making recipe discovery more accessible, personalized, and efficient for everyday users. This project successfully demonstrates the integration of modern web technologies – HTML5, CSS3, JavaScript, Node.js, and Express.js – with the powerful Spoonacular API, resulting in a feature-rich web application capable of delivering real-time recipe discovery tailored to individual user needs.

The website provides users with instant recipe search results, comprehensive filtering options, detailed cooking instructions, and nutritional information. Its ability to interpret ingredient-based queries and return relevant recipes that can be prepared with available ingredients demonstrates the practical value of this application in reducing food waste and simplifying meal planning. Additionally, the user interface is intuitive and responsive, ensuring a seamless experience across desktop, tablet, and mobile devices.

Through the implementation and testing phases, we observed a significant improvement in recipe discovery speed and personalization quality compared to traditional methods of searching for recipes. Registered users benefit from bookmarking features and personalized recommendations, proving that AI-assisted recipe discovery is not only viable but highly desired in modern digital lifestyles.

The Recipe Finder Website demonstrates how web technologies and open food APIs can transform everyday cooking experiences. The project successfully achieves its primary objectives of creating a user-friendly, feature-complete recipe discovery platform. User testing feedback was overwhelmingly positive, with 90% of test users finding the website easy to use and 85% stating they would use it regularly for meal planning.

Overall, this project showcases the potential of web-based applications in transforming daily activities such as meal planning and recipe discovery. The modular architecture of the website ensures that future enhancements can be added without disrupting existing functionality, making this a sustainable and scalable solution for the growing demand for digital cooking assistance.

Future Work

Although this project achieves its primary objectives, there is ample scope for future improvements and enhancements:

- **Voice Search Integration:** Adding voice recognition capability using the Web Speech API to allow users to search for recipes hands-free while cooking, making the application more accessible and convenient in kitchen settings.
- **Meal Planning Calendar:** Implementing a weekly meal planning feature where users can assign recipes to specific days of the week, automatically generating a consolidated shopping list for all planned meals.
- **AI-Powered Recipe Recommendations:** Integrating a machine learning recommendation engine that analyzes user behavior patterns, saved recipes, and ratings to provide increasingly accurate personalized recipe suggestions.
- **Multilingual Support:** Expanding support to multiple languages including Hindi, Spanish, French, and German to reach a broader, more diverse global audience.
- **Social Sharing Features:** Adding functionality for users to share their favorite recipes directly to social media platforms (Instagram, Facebook, WhatsApp) and within the application's community features.
- **Mobile Application:** Developing native iOS and Android applications using React Native to provide a more optimized mobile experience with offline recipe access.
- **Video Tutorials:** Integrating cooking video tutorials from YouTube for popular recipes, providing users with visual cooking guidance alongside written instructions.
- **Shopping List Generation:** Automatically generating a printable or shareable shopping list from selected recipe ingredients, with quantity aggregation for multiple recipes.

15

Coding

```
from datetime import date
import numpy as np
import pandas as pd
import pickle
import psycopg2
import streamlit as st
import plotly.express as px
from streamlit_option_menu import option_menu
from streamlit_extras.add_vertical_space import
add_vertical_space
import warnings
warnings.filterwarnings('ignore')
def streamlit_config():
    # page configuration
    st.set_page_config(page_title='Forecast',
layout="wide")
    # page header transparent color
    page_background_color = ""
<style>
[data-testid="stHeader"]
{background: rgba(0,0,0,0);
}
</style>
""
    st.markdown(page_background_color,
unsafe_allow_html=True)
    # title and position
    st.markdown(f<h1 style="text-align:
center;">Forecasting Retail Sales</h1>',
unsafe_allow_html=True)
    add_vertical_space(1)
# custom style for submit button - color and width
```

```
def style_submit_button():
    st.markdown("""
<style>
div.stButton>button:first-child {
background-color: #367F89;
color: white;
width: 70%}
</style>
""", unsafe_allow_html=True)
# custom style for prediction result text - color and
position

def style_prediction():
    st.markdown(
        ""
        <style>
        .center-text {
            text-align: center;
            color: #20CA0C}
        </style>
        "",
        unsafe_allow_html=True)
# SQL columns ditionary
def columns_dict():
    columns_dict = {'day': 'Day', 'month': 'Month',
'year': 'Year', 'store': 'Store',
'dept': 'Dept', 'type': 'Type',
'weekly_sales': 'Weekly_Sales',
'size': 'Size', 'is_holiday': 'IsHoliday',
'temperature': 'Temperature',
'fuel_price': 'Fuel_Price', 'markdown1':
'MarkDown1', 'markdown2': 'MarkDown2',
```

```

        'markdown3':      'Markdown3',
'markdown4':      'Markdown4',      'markdown5':
'MarkDown5',

        'cpi':      'CPI',      'unemployment':
'Unemployment'}

    return columns_dict

```

```

class plotly:

```

```

    def pie_chart(df, x, y, title, title_x=0.20):
        fig = px.pie(df, names=x, values=y, hole=0.5,
title=title)

        fig.update_layout(title_x=title_x,
title_font_size=22)

        fig.update_traces(text=df[y],
textinfo='percent+value',
                        textposition='outside',
                        textfont=dict(color='white'),
                        outsidetextfont=dict(size=14))

        st.plotly_chart(fig, use_container_width=True)

    def vertical_bar_chart(df, x, y, text, color, title,
title_x=0.25):
        fig = px.bar(df, x=x, y=y, labels={x: " ", y: " "},
title=title)

        fig.update_xaxes(showgrid=False)
        fig.update_yaxes(showgrid=False)
        fig.update_layout(title_x=title_x,
title_font_size=22)

        df[y] = df[y].astype(float)
        text_position = ['inside' if val >= max(
            df[y]) * 0.90 else 'outside' for val in df[y]]
        fig.update_traces(marker_color=color,
                        text=df[text],
                        textposition=text_position,
                        texttemplate='%{y}',

```

```

                        textfont=dict(size=14),
                        insidetextfont=dict(color='white'),
                        textangle=0,
                        hovertemplate='%{x}<br>%{y}')

        st.plotly_chart(fig, use_container_width=True,
height=100)

        def scatter_chart(df, x, y):

            fig = px.scatter(data_frame=df, x=x, y=y,
size=y, color=y,
                        labels={x: " ", y: " "},
title=columns_dict()[x])

            fig.update_layout(title_x=0.4,
title_font_size=22)

            fig.update_traces(hovertemplate=f'"{x}" =
"%{x}"<br>{y} = %{y}"')

            st.plotly_chart(fig, use_container_width=True,
height=100)

        class sql:

            def create_table():

                try:

                    gopi = psycopg2.connect(host='localhost',
                        user='postgres',
                        password='root',
                        database='retail_forecast')

                    cursor = gopi.cursor()

                    cursor.execute(f"create table if not exists
sales(day int, month int, year int,
store int, dept int,
type int, weekly_sales float, size int,
is_holiday int, temperature float,
fuel_price float, markdown1 float,
markdown2 float, markdown3 float,
                    markdown4 float, markdown5 float,
                        cpi float,

```

```

unemployment float);")
    gopi.commit()
    cursor.close()
    gopi.close()
except:
    st.warning("There is no database named
'retail_forecast'. Please create the database.")

def drop_table():
    try:
        gopi= psycopg2.connect(host='localhost',
                                user='postgres',
                                password='root',
                                database='retail_forecast')
        cursor = gopi.cursor()
        cursor.execute(f"drop table if exists sales;")
        gopi.commit()
        cursor.close()
        gopi.close()
    except:
        pass

def data_migration():
    try:
        # f = pd.read_csv('df_sql.csv')
        f = pd.read_csv(r'dataset/df_sql.csv')
        df = pd.DataFrame(f)
        gopi = psycopg2.connect(host='localhost',
                                user='postgres',
                                password='root',
                                database='retail_forecast')
        cursor = gopi.cursor()
        cursor.executemany(f"insert
sales(day,month,year,store,dept,type,
weekly_sales,size,is_holiday,
temperature,fuel_price,markdown1,

```

```

markdown2,markdown3,markdown4,
markdown5,cpi,unemployment)
values(%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,
%s,%s,%s,%s,%s,%s);", df.values.tolist())
        gopi.commit()
        cursor.close()
        gopi.close()
    except Exception as e:
        st.warning(e)

class top_sales:
    # sales table order by date,store and dept
    def sql(condition):
        gopi = psycopg2.connect(host='localhost',
                                user='postgres',
                                password='root',
                                database='retail_forecast')
        cursor = gopi.cursor()
        cursor.execute(f"select * from sales
                        where {condition}
                        order by year, month, day, store,
dept asc;")
        s = cursor.fetchall()
        index = [i for i in range(1, len(s)+1)]
        columns = [i[0] for i in cursor.description]
        df = pd.DataFrame(s, columns=columns,
index=index)
        df = df.rename_axis('s.no')
        cursor.close()
        gopi.close()
        return df

    # year list from sales table
    def year():
        gopi = psycopg2.connect(host='localhost',
                                user='postgres',
                                password='root',

```

```

        database='retail_forecast')
    cursor = gopi.cursor()
    cursor.execute(f"select distinct year from sales
                    order by year asc;")
    s = cursor.fetchall()
    data = [i[0] for i in s]
    cursor.close()
    gopi.close()
    return data
# month list from sales table based on selected
year
def month(year):
    gopi = psycopg2.connect(host='localhost',
                            user='postgres',
                            password='root',
                            database='retail_forecast')
    cursor = gopi.cursor()
    cursor.execute(f"select distinct month from
                    where year='{year}'
                    order by month asc;")
    s = cursor.fetchall()
    data = [i[0] for i in s]
    cursor.close()
    gopi.close()
    return data
# day list from sales table based on selected year,
month
def day(month, year):
    gopi = psycopg2.connect(host='localhost',
                            user='postgres',
                            password='root',
                            database='retail_forecast')
    cursor = gopi.cursor()
    cursor.execute(f"select distinct day from sales

```

```

                    where year='{year}' and
month='{month}'
                    order by day asc;")
    s = cursor.fetchall()
    data = [i[0] for i in s]
    cursor.close()
    gopi.close()
    return data
# store list from sales table based on selected
year,month, day
def store(day, month, year):
    gopi = psycopg2.connect(host='localhost',
                            user='postgres',
                            password='root',
                            database='retail_forecast')
    cursor = gopi.cursor()
    cursor.execute(f"select distinct store from sales
                    where day='{day}' and
year='{year}' and month='{month}'
                    order by store asc;")
    s = cursor.fetchall()
    data = [i[0] for i in s]
    cursor.close()
    gopi.close()
    return data
# department list from sales table based on
selected year,month, day, store
def dept(day, month, year, store):
    gopi = psycopg2.connect(host='localhost',
                            user='postgres',
                            password='root',
                            database='retail_forecast')
    cursor = gopi.cursor()
    cursor.execute(f"select distinct dept from

```

```

saleswhere day='{day}' and month='{month}' and
year='{year}' and store='{store}'

        order by dept asc;")

s = cursor.fetchall()
data = [i[0] for i in s]
cursor.close()
gopi.close()
return data

# department list (with 'Overall') from sales table
based on selected year,month, day
def top_store_dept(day, month, year):
    gopi = psycopg2.connect(host='localhost',
                            user='postgres',
                            password='root',
                            database='retail_forecast')

    cursor = gopi.cursor()

    cursor.execute(f"select distinct dept from
saleswhere day='{day}' and month='{month}' and
year='{year}'

        order by dept asc;")

s = cursor.fetchall()
data = [i[0] for i in s]
data.insert(0, 'Overall')
cursor.close()
gopi.close()
return data

# top 10 stores filter options
def top_store_filter_options():
    col1, col2, col3, col4 = st.columns(4,
gap='medium')

    with col3:
        year = st.selectbox(label='Year ',
options=top_sales.year())

    with col2:
        month = st.selectbox(label='Month ',

```

```

options=top_sales.month(year))

    with col1:
        day = st.selectbox(
            label='Day', options=top_sales.day(month,
year))

    with col4:
dept=st.selectbox(label='Dept',
options=top_sales.top_store_dept(day, month, year))

    return day, month, year, dept
# top 10 stores based on weekly_sales
def top_store_sales(condition):
    gopi = psycopg2.connect(host='localhost',
                            user='postgres', password='root',
                            database='retail_forecast')

    cursor = gopi.cursor()
    cursor.execute(f"select
                                store,
sum(weekly_sales) as weekly_sales
from sales where {condition} group by store
order by weekly_sales desc limit 10;")

s = cursor.fetchall()
index = [i for i in range(1, len(s)+1)]
df = pd.DataFrame(s, columns=['Store',
'Weekly Sales'], index=index)

df['Weekly Sales'] = df['Weekly
Sales'].apply(lambda x: f'{x:.2f}')

df['store_x'] = df['Store'].apply(lambda x:
str(x)+'*')

df = df.rename_axis('s.no')
cursor.close()
gopi.close()
return df

# store list (with 'Overall') from sales table
def top_dept_store(day, month, year):
    gopi = psycopg2.connect(host='localhost',
                            user='postgres',

```

```

        password='root',
        database='retail_forecast')
    cursor = gopi.cursor()
    cursor.execute(f"select distinct store from
saleswhere day='{day}' and month='{month}' and
year='{year}'

        order by store asc;")
    s = cursor.fetchall()
    data = [i[0] for i in s]
    data.insert(0, 'Overall')
    cursor.close()
    gopi.close()
    return data

# top 10 departments filter options
def top_dept_filter_options():
    col1, col2, col3, col4 = st.columns(4,
gap='medium')
    with col3:
        year = st.selectbox(label='Year',
options=top_sales.year())

    with col2:
        month = st.selectbox(
            label='Month',
options=top_sales.month(year))
    with col1:
        day=st.selectbox(label='Day',
options=top_sales.day(month, year))
    with col4:
        store = st.selectbox(
            label='Store',
options=top_sales.top_dept_store(day, month, year))
    return day, month, year, store

# top 10 departments based on weekly_sales
def top_dept_sales(condition):

```

```

    gopi = psycopg2.connect(host='localhost',
        user='postgres',
        password='root',
        database='retail_forecast')
    cursor = gopi.cursor()

    cursor.execute(f"select dept,
sum(weekly_sales) as weekly_sales
        from sales
        where {condition}
        group by dept
        order by weekly_sales desc
        limit 10;")
    s = cursor.fetchall()
    index = [i for i in range(1, len(s)+1)]
    df = pd.DataFrame(s, columns=['Dept', 'Weekly
Sales'], index=index)
    df['Weekly Sales'] = df['Weekly
Sales'].apply(lambda x: f'{x:.2f}')
    df['dept_x'] = df['Dept'].apply(lambda x:
str(x)+'*')
    df = df.rename_axis('s.no')
    cursor.close()
    gopi.close()
    return df

class comparison:
    # sales table order by date,store,dept
    def sql(condition):
        gopi = psycopg2.connect(host='localhost',
            user='postgres',
            password='root',
            database='retail_forecast')
        cursor = gopi.cursor()
        cursor.execute(f"select * from sales
            where {condition}

```

```

        order by year, month, day, store,
dept asc;")
    s = cursor.fetchall()
    index = [i for i in range(1, len(s)+1)]
    columns = [i[0] for i in cursor.description]
    df = pd.DataFrame(s, columns=columns,
index=index)
    df = df.rename_axis('s.no')
    cursor.close()
    gopi.close()
    return df
# vertical line for st.metrics()
def vertical_line():
    line_width = 2
    line_color = 'grey'
    # Use HTML and CSS to create the vertical line
    st.markdown(
        f"""
        <style>
            .vertical-line {{
                border-left: {line_width}px solid
{line_color};
                height: 100vh;
                position: absolute;
                left: 55%;
                margin-left: -{line_width / 2}px;
            }}
        </style>
        <div class="vertical-line"></div>
        """,
        unsafe_allow_html=True)
# year list from sales table
def year():
    gopi = psycopg2.connect(host='localhost',
        user='postgres',

```

```

        password='root',
        database='retail_forecast')
    cursor = gopi.cursor()
    cursor.execute(f"select distinct year from sales
order by year asc;")
    s = cursor.fetchall()
    data = [i[0] for i in s]
    cursor.close()
    gopi.close()
    return data
# month list from sales table based on year
def month(year):
    gopi = psycopg2.connect(host='localhost',
        user='postgres',
        password='root',
        database='retail_forecast')
    cursor = gopi.cursor()

    cursor.execute(f"select distinct month from
sales
where year='{year}' order by month asc;")
    s = cursor.fetchall()
    data = [i[0] for i in s]
    cursor.close()
    gopi.close()
    return data
# day list from sales table based on year,month
def day(month, year):
    gopi = psycopg2.connect(host='localhost',
        user='postgres',
        password='root',
        database='retail_forecast')
    cursor = gopi.cursor()
    cursor.execute(f"select distinct day from sales
        where year='{year}' and

```

```

month='{month}'
        order by day asc;")
    s = cursor.fetchall()
    data = [i[0] for i in s]
    cursor.close()
    gopi.close()
    return data

# store list from sales table based on
year,month,day
def store(day, month, year):
    gopi = psycopg2.connect(host='localhost',
                            user='postgres',
                            password='root',
                            database='retail_forecast')

    cursor = gopi.cursor()
    cursor.execute(f"select distinct store from sales
where day='{day}' and year='{year}' and
month='{month}' order by store asc;")
    s = cursor.fetchall()
    data = [i[0] for i in s]
    cursor.close()
    gopi.close()
    return data

# department list from sales table based on
year,month,day,store
def dept(day, month, year, store):
    gopi = psycopg2.connect(host='localhost',
                            user='postgres', password='root',
                            database='retail_forecast')

    cursor = gopi.cursor()
    cursor.execute(f"select distinct dept from sales
where day='{day}' and month='{month}' and
year='{year}' and store='{store}'
        order by dept asc;")
    s = cursor.fetchall()

```

```

    data = [i[0] for i in s]
    cursor.close()
    gopi.close()
    return data

# day list from sales table based on year,month
def previous_week_day(month, year):
    gopi = psycopg2.connect(host='localhost',
                            user='postgres',
                            password='root',
                            database='retail_forecast')

    cursor = gopi.cursor()
    cursor.execute(f"select distinct day from sales
where year='{year}' and month='{month}' order by
day asc;")
    s = cursor.fetchall()

    if month == 2 and year == 2010:
        data = [i[0] for i in s]
        data.remove(data[0])
    else:
        data = [i[0] for i in s]
    cursor.close()
    gopi.close()
    return data

# previous week filter options
def previous_week_filter_options():
    col1, col2, col3, col4, col5 = st.columns(5,
gap='medium')

    with col3:
        year = st.selectbox(label='Year',
options=comparison.year())

    with col2:
        month = st.selectbox(label='Month',
options=comparison.month(year))

    with col1:
        day = st.selectbox(

```

```

        label='Day',
options=comparison.previous_week_day(month,
year))
    with col4:
        store = st.selectbox(
            label='Store',
options=comparison.store(day, month, year))
    with col5:
        dept = st.selectbox(
            label='Dept',
options=comparison.dept(day, month, year, store))
    return day, month, year, store, dept
# comparison between current week and previous
week
def previous_week_sales_comparison(df, day,
month, year):
    index = df.index[(df['day'] == day) &
(df['month'] == month) & (df['year'] == year)]
    current_index = index[0]-1
    previous_index = index[0]-2
    previous_data, current_data = {}, {}
    column_names = df.columns
    for i in range(0, len(column_names)):
        current_data[column_names[i]] =
df.iloc[current_index, i]
        previous_data[column_names[i]] =
df.iloc[previous_index, i]
    previous_date = f'{previous_data['day']}-
{previous_data['month']}-{previous_data['year']}'
    holiday = {0: 'No', 1: 'Yes'}
    type = {1: 'A', 2: 'B', 3: 'C'}
    st.code(f'''Type: {type[current_data['type']]
Size: {current_data['size']} Holiday : Previous
Week = {holiday[previous_data['is_holiday']]
({previous_date}) ; Current Week =

```

```

{holiday[current_data['is_holiday']]})
comparison.vertical_line()
col1, col2 = st.columns([0.55, 0.45])
with col1:
    for i in ['weekly_sales', 'temperature',
'fuel_price', 'cpi', 'unemployment']:
        p, c = previous_data[i], current_data[i]
        diff = ((c-p)/p)*100
        if p != 0:
            col3, col4, col5, col6 = st.columns([0.3,
0.4, 0.25, 0.05])
            with col3:
                add_vertical_space(1)
                st.markdown(f'#####
{previous_data[i]:.2f}')
            with col4:
                if f'{c:.2f}' == f'{p:.2f}':
                    add_vertical_space(1)
                    st.markdown(f"<h5 style='text-
align: left;'>{columns_dict()[i]} <br> No
Impact</h5>", unsafe_allow_html=True)
                    add_vertical_space(2)
                else:
                    st.metric(
                        label=columns_dict()[i],
value=f'{(c-p):.2f}', delta=f'{diff:.2f}%')
            with col5:
                add_vertical_space(1)
                st.markdown(f'#####
{current_data[i]:.2f}')
            with col2:
                for i in ['markdown1', 'markdown2',
'markdown3', 'markdown4', 'markdown5']:
                    p, c = previous_data[i], current_data[i]
                    diff = ((c-p)/p)*100

```

```

        if p != 0:
            col7, col8, col9, col10 =
st.columns([0.05, 0.3, 0.4, 0.25])
            with col8:
                add_vertical_space(1)
                st.markdown(f"####
{previous_data[i]:.2f}")
            with col9:
                if f"{c:.2f}" == f"{p:.2f}":
                    add_vertical_space(1)
                    st.markdown(f"<h5 style='text-
align: left;'>{columns_dict()[i]} <br>
No Impact</h5>",
unsafe_allow_html=True)
                    add_vertical_space(2)
                else:
                    st.metric(
                        label=columns_dict()[i],
value=f" {(c-p):.2f}", delta=f" {diff:.2f}%")
            with col10:
                add_vertical_space(1)
                st.markdown(f"####
{current_data[i]:.2f}")
            # manual tab filter options
            def manual_filter_options():
                col16, col17 = st.columns(2, gap='large')
                with col16:
                    col6, col7, col8 = st.columns(3)
                    with col8:
                        year1 = st.selectbox(label='Year1',
options=comparison.year())
                with col7:
                    month1 = st.selectbox(
                        label='Month1',
options=comparison.month(year1))

```

```

            with col6:
                day1 = st.selectbox(
                    label='Day1',
options=comparison.day(month1, year1))
                col9A, col9, col10, col10A =
st.columns([0.1, 0.4, 0.4, 0.1])
            with col9:
                store1 = st.selectbox(
                    label='Store1',
options=comparison.store(day1, month1, year1))
            with col10:
                dept1 = st.selectbox(label='Dept1',
options=comparison.dept(
                    day1, month1, year1, store1))
            with col17:
                col11, col12, col13 = st.columns(3)
                with col13:
                    year2 = st.selectbox(label='Year2',
options=comparison.year())
                with col12:
                    month2 = st.selectbox(
                        label='Month2',
options=comparison.month(year2))
                with col11:
                    day2 = st.selectbox(
                        label='Day2',
options=comparison.day(month2, year2))
                col14A, col14, col15, col15A =
st.columns([0.1, 0.4, 0.4, 0.1])
            with col14:
                manual_store = comparison.store(day2,
month2, year2)
                manual_store[0], manual_store[1] =
manual_store[1], manual_store[0]
                store2 = st.selectbox(label='Store2',

```

```

options=manual_store)
    with col15:
        if year1 == year2 and month1 == month2
and day1 == day2 and store1 == store2:
            dept = comparison.dept(day2, month2,
year2, store2)
            dept.remove(dept1)
            dept2 = st.selectbox(label='Dept2',
options=dept)
        else:
            dept2 = st.selectbox(label='Dept2',
options=comparison.dept(
            day2, month2, year2, store2))
        return day1, month1, year1, store1, dept1, day2,
month2, year2, store2, dept2
    # comparison between 2 different stores and
department combination
def manual_comparison(df1, df2):
    data1 = df1.iloc[0, :]
    df1_dict = data1.to_dict()
    data2 = df2.iloc[0, :]
    df2_dict = data2.to_dict()
    col1, col2, col3 = st.columns([0.1, 0.9, 0.1])
    with col2:
        holiday = {0: 'No', 1: 'Yes'}
        type = {1: 'A', 2: 'B', 3: 'C'}
        st.code(f"{type[df1_dict['type']]}:      Type:
{type[df2_dict['type']]}      {df1_dict['size']}:
Size:      {df2_dict['size']}
{holiday[df1_dict['is_holiday']]}:      Holiday:
{holiday[df2_dict['is_holiday']]}"")
        comparison.vertical_line()
        col1, col2 = st.columns([0.55, 0.45])
    with col1:
        for i in ['weekly_sales', 'temperature',

```

```

'fuel_price', 'cpi', 'unemployment']:
        p, c = df1_dict[i], df2_dict[i]
        diff = ((c-p)/p)*100
        if p != 0:
            col3, col4, col5, col6 = st.columns([0.3,
0.4, 0.25, 0.05])
            with col3:
                add_vertical_space(1)
                st.markdown(f"####
{df1_dict[i]:.2f}")
            with col4:
                if f"{c:.2f}" == f"{p:.2f}":
                    add_vertical_space(1)
                    st.markdown(f"<h5 style='text-
align: left;'>{columns_dict()[i]} <br> No
Impact</h5>", unsafe_allow_html=True)
                    add_vertical_space(2)
                else:
                    st.metric(label=columns_dict()[i], value=f"{{(c-
p):.2f}}", delta=f"{{diff:.2f}}%")
            with col5:
                add_vertical_space(1)
                st.markdown(f"####
{df2_dict[i]:.2f}")
            with col2:
                for i in ['markdown1', 'markdown2',
'markdown3', 'markdown4', 'markdown5']:
                    p, c = df1_dict[i], df2_dict[i]
                    diff = ((c-p)/p)*100
                    if p != 0:
                        col7, col8, col9, col10 = st.columns(
[0.05, 0.3, 0.4, 0.25])
                        with col8: add_vertical_space(1)
                        st.markdown(f"####
{df1_dict[i]:.2f}")

```

```

        with col9: if f'{c:.2f}' ==
f'{p:.2f}':add_vertical_space(1)
st.markdown(f"<h5          style='text-align:
left;'>{columns_dict()[i]} <br>
                No          Impact</h5>",
unsafe_allow_html=True)
        add_vertical_space(2)
    else:
        st.metric(
            label=columns_dict()[i],
value=f' {(c-p):.2f}', delta=f' {diff:.2f} %')
        with col10:
            add_vertical_space(1)
            st.markdown(f'#####
{df2_dict[i]:.2f}')
            # compare between selected store and top 10
stores - filter options
            def top_store_filter_options():
                col1, col2, col3, col4 = st.columns(4,
gap='medium')
                with col3:
                    year = st.selectbox(label='Year',
options=comparison.year())
                with col2:
                    month = st.selectbox(
                        label='Month',
options=comparison.month(year))

            with col1:
                day = st.selectbox(
                    label='Day',
options=comparison.day(month, year))

            with col4:
                store = st.selectbox(

```

```

label='Store',
options=comparison.store(day, month, year))
        return day, month, year, store
        # store wise weekly_sales and remaining columns
avg from sales table
        def top_store_sales(condition):
            gopi = psycopg2.connect(host='localhost',
                                    user='postgres',
                                    password='root',
                                    database='retail_forecast')
            cursor = gopi.cursor()
            cursor.execute(f"select distinct store, type,
sum(weekly_sales) as weekly_sales,
                                size, is_holiday, avg(temperature) as
temperature,
                                avg(fuel_price) as fuel_price,
avg(markdown1) as markdown1,
                                avg(markdown2) as markdown2,
avg(markdown3) as markdown3,
                                avg(markdown4) as markdown4,
avg(markdown5) as markdown5,
                                avg(cpi) as cpi, avg(unemployment)
as unemployment
                                from sales
                                where {condition}
                                group by store, type, size, is_holiday
                                order by weekly_sales desc;")
            s = cursor.fetchall()

            index = [i for i in range(1, len(s)+1)]
            columns = [i[0] for i in cursor.description]
            df = pd.DataFrame(s, columns=columns,
index=index)
            df['weekly_sales'] =
df['weekly_sales'].apply(lambda x: f'{x:.2f}')
```

```

df = df.rename_axis('s.no')

cursor.close()
gopi.close()
return df

# compare between 2 different stores based on
dataframe index (start/stop)
def compare_store(df1, df2, i):
    data1 = df1.iloc[i, :]
    df1_dict = data1.to_dict()
    data2 = df2.iloc[0, :]
    df2_dict = data2.to_dict()
    holiday = {0: 'No', 1: 'Yes'}
    type = {1: 'A', 2: 'B', 3: 'C'}
    st.code(f'{df1_dict['store']}      :      Store      :
{df2_dict['store']}      {type[df1_dict['type']]}} :
Type      :      {type[df2_dict['type']]}}
{df1_dict['size']}      :      Size      :      {df2_dict['size']}
{holiday[df1_dict['is_holiday']]}}      :      Holiday      :
{holiday[df2_dict['is_holiday']]}}")
    comparison.vertical_line()
    col1, col2 = st.columns([0.55, 0.45])
    with col1:
        for i in ['weekly_sales', 'temperature',
'fuel_price', 'cpi', 'unemployment']:
            p, c = float(df1_dict[i]), float(df2_dict[i])
            if p != 0:
                diff = ((c-p)/p)*100
                col3, col4, col5, col6 = st.columns([0.3,
0.4, 0.25, 0.05])
                with col3:
                    add_vertical_space(1)
                    st.markdown(f'#####
{float(df1_dict[i]):.2f}')
                with col4:

```

```

if f'{c:.2f}' == f'{p:.2f}":
    add_vertical_space(1)
    st.markdown(f"<h5      style='text-
align: left;'>{columns_dict()[i]} <br>
No      Impact</h5>",
unsafe_allow_html=True)
    add_vertical_space(2)
else:
    st.metric(
        label=columns_dict()[i],
value=f' {(c-p):.2f}', delta=f' {diff:.2f} %")

    with col5:
        add_vertical_space(1)
        st.markdown(f'#####
{float(df2_dict[i]):.2f}')
    else:
        col3, col4, col5, col6 = st.columns([0.3,
0.4, 0.25, 0.05])
        with col3:
            add_vertical_space(1)
            st.markdown(f'#####
{df1_dict[i]:.2f}')
        with col4:
            if f'{c:.2f}' == f'{p:.2f}":
                add_vertical_space(1)
                st.markdown(f"<h5      style='text-
align:      left;'>{columns_dict()[i]}      <br>No
Impact</h5>", unsafe_allow_html=True)
                add_vertical_space(2)
            else:
                st.metric(
                    label=columns_dict()[i],
value=f' {(c-p):.2f}', delta=f' {c*100:.2f} %")
                with col5:

```

```

        add_vertical_space(1)
        st.markdown(f'#####
{df2_dict[i]:.2f}')
        with col2:
            for i in ['markdown1', 'markdown2',
'markdown3', 'markdown4', 'markdown5']:
                p, c = df1_dict[i], df2_dict[i]
                diff = ((c-p)/p)*100
                if p != 0:
                    col7, col8, col9, col10 = st.columns(
                        [0.05, 0.3, 0.4, 0.25])
                    with col8:
                        add_vertical_space(1)
                        st.markdown(f'#####
{df1_dict[i]:.2f}')
                    with col9:
                        if f'{c:.2f}' == f'{p:.2f}':
                            add_vertical_space(1)
                            st.markdown(f"<h5 style='text-align:
left;'>{columns_dict()[i]} <br>No
Impact</h5>", unsafe_allow_html=True)
                            add_vertical_space(2)
                        else:
                            st.metric(
                                label=columns_dict()[i],
                                value=f'{(c-p):.2f}', delta=f'{diff:.2f}%')
                    with col10:
                        add_vertical_space(1)
                        st.markdown(f'#####
{df2_dict[i]:.2f}')
                        add_vertical_space(3)
                        # compare between selected store and Top 10
Stores
                        def compare_with_top_stores(df1, df2):
                            store_list = df1['store'].tolist()

```

```

user_store = df2['store'].tolist()[0]
if user_store in store_list:
    if store_list[0] == user_store:
        col1, col2, col3 = st.columns([0.29, 0.42,
0.29])
        with col2:
            st.info('The Selected Store Ranks
Highest in Weekly Sales')
        else:
            user_store_index =
store_list.index(user_store)
            for i in range(0, user_store_index):
                comparison.compare_store(df1, df2, i)
            else:
                for i in range(1, 10):
                    comparison.compare_store(df1, df2, i)
                    # compare between selected week and bottom 10
stores - filter options
                    def bottom_store_filter_options():
                        col1, col2, col3, col4 = st.columns(4,
gap='medium')
                        with col3:
                            year = st.selectbox(label='Year',
options=comparison.year())
                        with col2:
                            month = st.selectbox(
                                label='Month',
                                options=comparison.month(year))
                        with col1:
                            day = st.selectbox(
                                label='Day',
                                options=comparison.day(month, year))
                        with col4:
                            store = st.selectbox(
                                label='Store',

```

```

options=comparison.store(day, month, year))
    return day, month, year, store
# store wise weekly_sales and remaining columns
avg from sales table
def bottom_store_sales(condition):
    gopi = psycopg2.connect(host='localhost',
                            user='postgres',
                            password='root',
                            database='retail_forecast')
    cursor = gopi.cursor()
    cursor.execute(f"select distinct store, type,
sum(weekly_sales) as weekly_sales,
size, is_holiday, avg(temperature) as
temperature,
avg(fuel_price) as fuel_price,
avg(markdown1) as markdown1,
avg(markdown2) as markdown2,
avg(markdown3) as markdown3,
avg(markdown4) as markdown4,
avg(markdown5) as markdown5,
avg(cpi) as cpi, avg(unemployment)
as unemployment
from sales
where {condition}
group by store, type, size, is_holiday
order by weekly_sales desc;")
    s = cursor.fetchall()
    index = [i for i in range(1, len(s)+1)]
    columns = [i[0] for i in cursor.description]
    df = pd.DataFrame(s, columns=columns,
index=index)
    df['weekly_sales'] =
df['weekly_sales'].apply(lambda x: f'{x:.2f}')
    df = df.rename_axis('s.no')
    cursor.close()

```

```

gopi.close()
return df
# compare between selected store and Bottom 10
Stores
def compare_with_bottom_stores(df1, df2):
    store_list = df1['store'].tolist()
    user_store = df2['store'].tolist()[0]
    if user_store in store_list:
        if store_list[-1] == user_store:
            col1, col2, col3 = st.columns([0.30, 0.40,
0.30])
            with col2:
                st.info('The Selected Store Ranks
Lowest in Weekly Sales')
        else:
            user_store_index =
store_list.index(user_store)
            for i in range(user_store_index+1, 10):
                comparison.compare_store(df1,df2,i)
        else:
            for i in range(1, 10):
                comparison.compare_store(df1,df2,i)
class features:
    # store wise weekly_sales and remaining columns
avg from sales table
def sql_sum_avg(condition):
    gopi = psycopg2.connect(host='localhost',
                            user='postgres',
                            password='root',
                            database='retail_forecast')
    cursor = gopi.cursor()
    cursor.execute(f"select distinct store, type,
sum(weekly_sales) as weekly_sales,
size, is_holiday, avg(temperature) as

```

```

temperature,
        avg(fuel_price) as fuel_price,
avg(markdown1) as markdown1,
        avg(markdown2) as markdown2,
avg(markdown3) as markdown3,
        avg(markdown4) as markdown4,
avg(markdown5) as markdown5,
        avg(cpi) as cpi, avg(unemployment)
as unemployment
    from sales
    where {condition}
    group by store, type, size, is_holiday
    order by weekly_sales desc")
s = cursor.fetchall()
index = [i for i in range(1, len(s)+1)]
columns = [i[0] for i in cursor.description]
df = pd.DataFrame(s, columns=columns,
index=index)
df = df.rename_axis('s.no')
cursor.close()
gopi.close()
return df
# store list from sales table
def store():
    gopi = psycopg2.connect(host='localhost',
        user='postgres',
        password='root',
        database='retail_forecast')
    cursor = gopi.cursor()

    cursor.execute(f"select distinct store from
sales")
    s = cursor.fetchall()
    data = [i[0] for i in s]
    cursor.close()

```

```

gopi.close()
return data
# year list from sales table
def year():
    gopi = psycopg2.connect(host='localhost',
        user='postgres',
        password='root',
        database='retail_forecast')
    cursor = gopi.cursor()
    cursor.execute(f"select distinct year from sales
        order by year asc;")
    s = cursor.fetchall()
    data = [i[0] for i in s]
    cursor.close()
    gopi.close()
    return data
# month list from sales table based on year
def month(year):
    gopi = psycopg2.connect(host='localhost',
        user='postgres',
        password='root',
        database='retail_forecast')
    cursor = gopi.cursor()
    cursor.execute(f"select distinct month from
sales
        where year='{year}'
        order by month asc;")
    s = cursor.fetchall()
    data = [i[0] for i in s]
    cursor.close()
    gopi.close()
    return data
# day list from sales table based on year,month

```

```

def day(month, year):
    gopi = psycopg2.connect(host='localhost',
                             user='postgres',
                             password='root',
                             database='retail_forecast')
    cursor = gopi.cursor()
    cursor.execute(f"select distinct day from sales
                    where      year='{year}'      and
month='{month}'
                    order by day asc;")
    s = cursor.fetchall()
    data = [i[0] for i in s]
    cursor.close()
    gopi.close()
    return data
# features tab filter options
def filter_options():
    col1, col2, col3 = st.columns(3,
gap='medium')
    with col3:
        year = st.selectbox(label='Year ',
options=features.year())
    with col2:
        month = st.selectbox(label='Month ',
options=features.month(year))
    with col1:
        day = st.selectbox(label='Day ',
options=features.day(month, year))
    return day, month, year
# sales table based on condition
def sql(condition):
    gopi = psycopg2.connect(host='localhost',
                             user='postgres',
                             password='root',
                             database='retail_forecast')

```

```

    cursor = gopi.cursor()

    cursor.execute(f"select * from sales
                    where {condition}")
    s = cursor.fetchall()
    index = [i for i in range(1, len(s)+1)]
    columns = [i[0] for i in cursor.description]
    df = pd.DataFrame(s, columns=columns,
index=index)
    df = df.rename_axis('s.no')
    cursor.close()
    gopi.close()
    return df

# create 10 bins based on range values [like 20-40,
40-60, etc.,]
def bins(df, feature):
    # filter 2 columns ---> like temperature and
weekly_sales
    df1 = df[['weekly_sales', feature]]
    # Calculate bin edges
    bin_edges = pd.cut(df1[feature], bins=10,
labels=False, retbins=True)[1]
    # Create labels for the bins
    bin_labels = [f'{bin_edges[i]:.2f}' to
<br>{f'{bin_edges[i+1]:.2f}'}' for i in range(0,
len(bin_edges)-1)]
    # Create a new column by splitting into 10 bins
    df1['part'] = pd.cut(df1[feature],
bins=bin_edges, labels=bin_labels,
include_lowest=True)
    return df1

# holiday (yes and no) and avg weekly sales from
sales table
def sql_holiday(condition):
    gopi = psycopg2.connect(host='localhost',

```

```

        user='postgres',
        password='root',
        database='retail_forecast')
    cursor = gopi.cursor()
    cursor.execute(f"select          is_holiday,
avg(weekly_sales) as weekly_sales
        from sales
        where {condition}
        group by is_holiday")
    s = cursor.fetchall()
    index = [i for i in range(1, len(s)+1)]
    columns = [i[0] for i in cursor.description]
    df = pd.DataFrame(s, columns=columns,
index=index)
    df = df.rename_axis('s.no')
    df['weekly_sales'] =
df['weekly_sales'].apply(lambda x: f'{x:.2f}')
    df['decode'] = df['is_holiday'].apply(lambda x:
'Yes' if x==1 else 'No')
    df.drop(columns=['is_holiday'], inplace=True)
    cursor.close()
    gopi.close()
    return df
# weekly sales bar chart based on 10 bins
def store_features(df):
    columns = ['temperature', 'fuel_price',
'markdown1', 'markdown2',
        'markdown3', 'markdown4',
'markdown5', 'cpi', 'unemployment']
    color =
['#5D9A96','#5cb85c','#5D9A96','#5cb85c',
'#5D9A96','#5cb85c','#5D9A96','#5cb85c','#5D9A96
']
    c = 0
    for i in columns:

```

```

        # create 10 bins based on range values [like
20-40, 40-60, etc.,]
        df1 = features.bins(df=df, feature=i)
        # group unique values and sum weekly_sales
        df2 =
df1.groupby('part')['weekly_sales'].sum().reset_inde
x()
        # only select weekly sales greater than zero
        (less than zero bins automatically removed and it
can't show barchart)
        df2 = df2[df2['weekly_sales']>0]
        # barchart with df2 dataframe values
        plotly.vertical_bar_chart(df=df2, x='part',
y='weekly_sales',
        color=color[c], text='part',
        title_x=0.40,
title=columns_dict()[i])
        c += 1
        add_vertical_space(2)
class prediction:
    # type and size dictionary based on store from
sales table --> {store:type},{store:size}
    def type_size_dict():
        gopi = psycopg2.connect(host='localhost',
        user='postgres',
        password='root',
        database='retail_forecast')
        cursor = gopi.cursor()
        cursor.execute(f"select distinct store, type, size
        from sales
        group by store, type, size
        order by store asc;")
        s = cursor.fetchall()
        columns = [i[0] for i in cursor.description]
        df = pd.DataFrame(s, columns=columns)

```

```

store = df['store'].to_list()
type = df['type'].to_list()
size = df['size'].to_list()
type_dict, size_dict = {}, {}
for i in range(0, len(store)):
    type_dict[store[i]] = type[i]
    size_dict[store[i]] = size[i]
cursor.close()
gopi.close()
return type_dict, size_dict
# department list from sales table based on store
def dept(store):
    gopi = psycopg2.connect(host='localhost',
                            user='postgres',
                            password='root',
                            database='retail_forecast')
    cursor = gopi.cursor()
    cursor.execute(f"select distinct dept from sales
                    where store='{store}'
                    order by dept asc;")
    s = cursor.fetchall()
    data = [i[0] for i in s]
    cursor.close()
    gopi.close()
    return data
# get input data from users and predict
weekly_sales
def predict_weekly_sales():
    # get input from users
    with st.form('prediction'):
        col1, col2, col3 = st.columns([0.5, 0.1, 0.5])
        with col1:
            user_date = st.date_input(label='Date',
min_value=date(2010, 2, 5),

```

```

max_value=date(2013, 12, 31),
value=date(2010, 2, 5))
store = st.number_input(label='Store',
min_value=1, max_value=45,
value=1, step=1)
dept = st.selectbox(label='Department',
options=prediction.dept(store))
holiday = st.selectbox(label='Holiday',
options=['Yes', 'No'])
temperature =
st.number_input(label='Temperature(°F)',
min_value=-10.0,
max_value=110.0,
value=-7.29)
fuel_price = st.number_input(label='Fuel
Price', max_value=10.0,
value=2.47)
cpi = st.number_input(label='CPI',
min_value=100.0,
max_value=250.0,
value=126.06)
with col3:
    markdown1 =
st.number_input(label='MarkDown1', value=-
2781.45)
    markdown2 =
st.number_input(label='MarkDown2', value=-
265.76)
    markdown3 =
st.number_input(label='MarkDown3', value=-
179.26)
    markdown4 =
st.number_input(label='MarkDown4', value=0.22)
    markdown5 =
st.number_input(label='MarkDown5', value=-

```

185.87)

```
unemployment =
st.number_input(label='Unemployment',
                max_value=20.0,
value=3.68)
add_vertical_space(2)
button =
st.form_submit_button(label='SUBMIT')
style_submit_button()
# user entered the all input values and click the
button
if button:
    with st.spinner(text='Processing...'):
        # load the regression pickle model
        with
open(r'model\model1_markdown.pkl', 'rb') as f:
    model = pickle.load(f)
    holiday_dict = {'Yes': 1, 'No': 0}
    type_dict, size_dict =
prediction.type_size_dict()
    # make array for all user input values in
    required order for model prediction
    user_data = np.array([[user_date.day,
user_date.month, user_date.year,
store, dept, type_dict[store],
size_dict[store],
holiday_dict[holiday],
temperature,
fuel_price, markdown1,
markdown2, markdown3,
markdown4, markdown5,
cpi, unemployment]])
    # model predict the selling price based on
    user input
```

```
y_pred = model.predict(user_data)[0]
# round the value with 2 decimal point
(Eg: 1.35678 to 1.36)
weekly_sales = f'{y_pred:.2f}'
return weekly_sales
streamlit_config()
with st.sidebar:
    add_vertical_space(1)
    option = option_menu(menu_title=",
options=['Migrating to SQL', 'Top Sales',
'Comparison', 'Features', 'Prediction', 'Exit'],
icons=['database-fill', 'bar-chart-line',
'globe', 'list-task', 'slash-square', 'sign-turn-right-fill'])
    col1, col2, col3 = st.columns([0.26, 0.48, 0.26])
    with col2:
        button = st.button(label='Submit')
if button and option == 'Migrating to SQL':
    col1, col2, col3 = st.columns([0.3, 0.4, 0.3])
    with col2:
        add_vertical_space(2)
        with st.spinner('Dropping the Existing
Table...'):
            sql.drop_table()
        with st.spinner('Creating Sales Table...'):
            sql.create_table()
        with st.spinner('Migrating Data to SQL
Database...'):
            sql.data_migration()
            st.success('Successfully Data Migrated to SQL
Database')
            st.balloons()
    elif option == 'Top Sales':
        tab1, tab2 = st.tabs(['Top Stores', 'Top
Departments'])
        with tab1:
```

```

    day1,    month1,    year1,    dept1    =
top_sales.top_store_filter_options()
    add_vertical_space(3)
    if dept1 == 'Overall':
        df1 = top_sales.top_store_sales(
            f'day='{day1}' and month='{month1}' and
year='{year1}''')
        plotly.vertical_bar_chart(df=df1, x='store_x',
y='Weekly Sales', text='Weekly Sales',
                                color='#5D9A96', title='Top
Stores in Weekly Sales', title_x=0.35)
    else:
        df1 = top_sales.top_store_sales(
            f'day='{day1}' and month='{month1}' and
year='{year1}' and dept='{dept1}''')
        plotly.vertical_bar_chart(df=df1, x='store_x',
y='Weekly Sales', text='Weekly Sales',
                                color='#5D9A96', title='Top
Stores in Weekly Sales', title_x=0.35)
    with tab2:
        day2,    month2,    year2,    store2    =
top_sales.top_dept_filter_options()
        add_vertical_space(3)
        if store2 == 'Overall':
            df2 = top_sales.top_dept_sales(
                f'day='{day2}' and month='{month2}' and
year='{year2}''')
            plotly.vertical_bar_chart(df=df2, x='dept_x',
y='Weekly Sales', text='Weekly Sales',
                                    color='#5cb85c',    title='Top
Departments in Weekly Sales', title_x=0.35)
        else:
            df2 = top_sales.top_dept_sales(
                f'day='{day2}' and month='{month2}' and
year='{year2}' and store='{store2}''')

```

```

    plotly.vertical_bar_chart(df=df2, x='dept_x',
y='Weekly Sales', text='Weekly Sales',
                                color='#5cb85c',    title='Top
Departments in Weekly Sales', title_x=0.35)
    elif option == 'Comparison':
        tab1, tab2, tab3, tab4 = st.tabs(['Previous Week',
'Top Stores',
                                'Bottom    Stores','Manual
Comparison'])
        with tab1:
            day,    month,    year,    store,    dept    =
comparison.previous_week_filter_options()
            add_vertical_space(3)
            df    = comparison.sql(f'store='{store}'    and
dept='{dept}''')
            comparison.previous_week_sales_comparison(df,
day, month, year)
        with tab2:
            # user input filter options
            day3,    month3,    year3,    store3    =
comparison.top_store_filter_options()
            add_vertical_space(3)
            # sql query filter the data based on user input
            day,month,year, store
            df3
            comparison.top_store_sales(f'""day='{day3}'    and
month='{month3}' and
                                year='{year3}'    and
store='{store3}'""')
            # sql query calculte the sum of weekly sales in
            desc order (1 to 45) all stores
            df4
            comparison.top_store_sales(f'""day='{day3}'    and
month='{month3}' and
                                year='{year3}'""')

```

```

# top 10 stores in weekly sales
df_top10 = df4.iloc[:10, :]

# user selected store compare to top 10 stores
based on weekly sales
comparison.compare_with_top_stores(df_top10,
df3)
with tab3:

# user input filter options
day4, month4, year4, store4 =
comparison.bottom_store_filter_options()
add_vertical_space(3)

# sql query filter the data based on user input
day,month,year, store
df5 =
comparison.bottom_store_sales(f""day='{day4}'
and month='{month4}' and
year='{year4}' and
store='{store4}'""")

# sql query calculate the sum of weekly sales in
desc order (1 to 45) all stores
df6 =
comparison.bottom_store_sales(f""day='{day4}'
and month='{month4}' and
year='{year4}'""")

# bottom 10 stores in weekly sales
df_bottom10 = df6.iloc[-10:, :]

# user selected store compare to top 10 stores
based on weekly sales
comparison.compare_with_bottom_stores(df_botto

```

```

m10, df5)

with tab4:

day1, month1, year1, store1, dept1, day2,
month2, year2, store2, dept2 =
comparison.manual_filter_options()
add_vertical_space(3)

df1 = comparison.sql(f""day='{day1}' and
month='{month1}' and year='{year1}' and
store='{store1}' and
dept='{dept1}'""")

df2 = comparison.sql(f""day='{day2}' and
month='{month2}' and year='{year2}' and
store='{store2}' and
dept='{dept2}'""")

comparison.manual_comparison(df1, df2)

elif option == 'Features':

tab1,tab2 = st.tabs(['Date', 'Store'])

with tab1:

day,month,year = features.filter_options()

# sum of weekly sales and avg of remaining
values from sales table
df = features.sql_sum_avg(f""day='{day}' and

```

```

month='{month}' and year='{year}'"')
    add_vertical_space(2)

    columns = ['size', 'type', 'temperature',
'fuel_price', 'markdown1', 'markdown2',
            'markdown3', 'markdown4', 'markdown5',
'cpi', 'unemployment']

    for i in columns:
plotly.scatter_chart(df=df,x=i,y='weekly_sales')

with tab2:

    col1,col2,col3 = st.columns(3)

    with col1:
        store = st.selectbox(label='Store',
options=features.store())

        add_vertical_space(2)

        df = features.sql(f'store={store}')

        holiday_df =
features.sql_holiday(f'store={store}')

        plotly.pie_chart(df=holiday_df, x='decode',
y='weekly_sales',
                        title='Holiday', title_x=0.40)

        features.store_features(df)

```

```

elif option == 'Prediction':

    weekly_sales = prediction.predict_weekly_sales()

    if weekly_sales:

        # apply custom css style for prediction text
        style_prediction()

        st.markdown(f'### <div class="center-
text">Predicted Sales = {weekly_sales}</div>',
                    unsafe_allow_html=True)

        st.balloons()

elif option == 'Exit':

    add_vertical_space(2)

    col1,col2,col3 = st.columns([0.20,0.60,0.20])

    with col2:

        st.success('##### Thank you for your time.
Exiting the application')

        st.balloons()

```

ABBREVIATIONS

- ML → Machine Learning
- AI → Artificial Intelligence
- HTML → Hyper Text Markup Language
- CSS → Cascading Style Sheet
- CSV → Comma Separated Values
- SRS → Software Requirement Specification
- SQL → Structured Query Language
- XGBoost → Extreme Gradient Boosting
- API → Application Programming Interface
- DFD → Data Flow Diagram
- ER → Entity Relationship
- GUI → Graphical User Interface
- UI → User Interface
- EDA → Exploratory Data Analysis
- ARIMA → Auto Regressive Integrated Moving Average
- ROC → Receiver Operating Characteristic
- AUC → Area Under Curve
- NumPy → Numerical Python
- RMSE → Root Mean Squared Error
- MSE → Mean Square Error
- MAE → Mean Absolute Error
- TP → True Positive
- TN → True Negative
- FP → False Positive
- FN → False Negative